

AstraLog-HPC

Generated by Doxygen 1.17.0

1 Directory Hierarchy	1
1.1 Directories	1
2 Namespace Index	3
2.1 Namespace List	3
3 Class Index	5
3.1 Class List	5
4 File Index	7
4.1 File List	7
5 Directory Documentation	9
5.1 src Directory Reference	9
6 Namespace Documentation	11
6.1 astralog Namespace Reference	11
6.1.1 Typedef Documentation	13
6.1.1.1 TokenViewMap	13
6.1.2 Enumeration Type Documentation	14
6.1.2.1 BatchStrategy	14
6.1.2.2 CompOp	14
6.1.2.3 Priority	14
6.1.2.4 RuleType	14
6.1.3 Function Documentation	15
6.1.3.1 assign_sensor_tokens()	15
6.1.3.2 build_inverse_token_map()	15
6.1.3.3 build_line_offsets()	16
6.1.3.4 build_per_sensor_sequences()	16
6.1.3.5 build_timestamp_groups()	17
6.1.3.6 build_token_view_map()	17
6.1.3.7 evaluate_comparison()	18
6.1.3.8 evaluate_correlations()	18
6.1.3.9 evaluate_rules_parallel()	19
6.1.3.10 evaluate_stateful()	20
6.1.3.11 evaluate_step_diff()	21
6.1.3.12 evaluate_threshold()	22
6.1.3.13 format_alarm_lines()	22
6.1.3.14 format_nominal_line()	23
6.1.3.15 format_value()	24
6.1.3.16 generate_output_parallel()	24
6.1.3.17 get_line()	26
6.1.3.18 load_rules()	27
6.1.3.19 load_sensors_from_yaml()	28

6.1.3.20 lookup_sensor_token()	28
6.1.3.21 parse_csv_parallel()	29
6.1.3.22 parse_value()	30
6.1.3.23 prepare_rules()	30
6.1.3.24 priority_to_string()	31
6.1.3.25 process_pipeline()	31
6.1.3.26 ruletype_to_string()	32
6.1.3.27 sort_by_priority()	32
6.1.3.28 sort_violations_for_output()	33
6.1.3.29 strategy_to_string()	33
6.1.3.30 string_to_compop()	34
6.1.3.31 string_to_priority()	34
6.1.3.32 string_to_ruletype()	34
6.1.3.33 string_to_strategy()	35
6.1.3.34 tokenize_csv_line()	35
6.1.3.35 validate_schema()	36
6.2 astralog::detail Namespace Reference	36
6.2.1 Function Documentation	36
6.2.1.1 read_file_to_string()	36
7 Class Documentation	37
7.1 astralog::BatchAccumulator Class Reference	37
7.1.1 Detailed Description	38
7.1.2 Constructor & Destructor Documentation	38
7.1.2.1 BatchAccumulator()	38
7.1.3 Member Function Documentation	38
7.1.3.1 add_record()	38
7.1.3.2 batch_count()	39
7.1.3.3 buffered_count()	39
7.1.3.4 config()	39
7.1.3.5 flush()	40
7.1.3.6 flush_remaining()	40
7.1.3.7 should_flush()	41
7.2 astralog::BatchConfig Struct Reference	41
7.2.1 Detailed Description	42
7.2.2 Constructor & Destructor Documentation	42
7.2.2.1 BatchConfig()	42
7.2.3 Member Data Documentation	42
7.2.3.1 batch_output_dir	42
7.2.3.2 count_threshold	42
7.2.3.3 strategy	42
7.2.3.4 time_threshold_ms	43

7.2.3.5 write_audit_files	43
7.3 astralog::detail::MiniJsonParser::JsonValue Struct Reference	43
7.3.1 Member Enumeration Documentation	44
7.3.1.1 Type	44
7.3.2 Member Function Documentation	44
7.3.2.1 as_int()	44
7.3.2.2 as_number()	44
7.3.2.3 as_string()	44
7.3.2.4 find()	45
7.3.3 Member Data Documentation	45
7.3.3.1 arr_val	45
7.3.3.2 num_val	45
7.3.3.3 obj_val	45
7.3.3.4 str_val	45
7.3.3.5 type	45
7.4 astralog::MappedFile Struct Reference	45
7.4.1 Detailed Description	46
7.4.2 Constructor & Destructor Documentation	46
7.4.2.1 ~MappedFile()	46
7.4.3 Member Function Documentation	46
7.4.3.1 open()	46
7.4.4 Member Data Documentation	47
7.4.4.1 data	47
7.4.4.2 fd	47
7.4.4.3 size	47
7.5 astralog::detail::MiniJsonParser Class Reference	47
7.5.1 Member Function Documentation	48
7.5.1.1 parse()	48
7.6 astralog::ParsedRecord Struct Reference	48
7.6.1 Constructor & Destructor Documentation	49
7.6.1.1 ParsedRecord()	49
7.6.2 Member Data Documentation	49
7.6.2.1 sensor_token	49
7.6.2.2 timestamp	49
7.6.2.3 value	49
7.7 astralog::PipelineState Struct Reference	50
7.7.1 Detailed Description	50
7.7.2 Member Function Documentation	50
7.7.2.1 initialize()	50
7.7.3 Member Data Documentation	51
7.7.3.1 initialized	51
7.7.3.2 sensor_states	51

7.8 astralog::PreparedRules Struct Reference	51
7.8.1 Detailed Description	52
7.8.2 Member Data Documentation	52
7.8.2.1 correlation_rules	52
7.8.2.2 sensor_rules	52
7.8.2.3 stateful_count	52
7.8.2.4 step_diff_count	52
7.8.2.5 threshold_count	52
7.9 astralog::ProcessingResult Struct Reference	53
7.9.1 Detailed Description	53
7.9.2 Member Data Documentation	54
7.9.2.1 alarm_lines	54
7.9.2.2 eval_ms	54
7.9.2.3 num_anomalies	54
7.9.2.4 num_nominal	54
7.9.2.5 num_timestamps	54
7.9.2.6 total_ms	54
7.9.2.7 valid_lines	54
7.10 astralog::TimestampGroup::Reading Struct Reference	55
7.10.1 Member Data Documentation	55
7.10.1.1 sensor_token	55
7.10.1.2 value	55
7.11 astralog::RuleDefinition Struct Reference	56
7.11.1 Detailed Description	57
7.11.2 Constructor & Destructor Documentation	57
7.11.2.1 RuleDefinition()	57
7.11.3 Member Data Documentation	57
7.11.3.1 consecutive_measurements	57
7.11.3.2 logic	57
7.11.3.3 op	57
7.11.3.4 priority	57
7.11.3.5 rule_id	57
7.11.3.6 sensor_id	58
7.11.3.7 sensor_token_idx	58
7.11.3.8 sub_rules	58
7.11.3.9 threshold	58
7.11.3.10 type	58
7.12 astralog::RuleViolation Struct Reference	59
7.12.1 Detailed Description	60
7.12.2 Constructor & Destructor Documentation	60
7.12.2.1 RuleViolation() [1/2]	60
7.12.2.2 RuleViolation() [2/2]	60

7.12.3 Member Function Documentation	61
7.12.3.1 add()	61
7.12.3.2 empty()	61
7.12.3.3 sensor_token_at()	61
7.12.3.4 size()	61
7.12.3.5 value_at()	61
7.12.4 Member Data Documentation	61
7.12.4.1 count	61
7.12.4.2 INLINE_CAPACITY	62
7.12.4.3 rule	62
7.12.4.4 sensor_tokens_extra	62
7.12.4.5 sensor_tokens_inline	62
7.12.4.6 values_extra	62
7.12.4.7 values_inline	62
7.13 astralog::SensorRuleSet Struct Reference	62
7.13.1 Detailed Description	63
7.13.2 Member Data Documentation	63
7.13.2.1 stateful	63
7.13.2.2 step_diff	63
7.13.2.3 threshold	63
7.14 astralog::SensorState Struct Reference	63
7.14.1 Detailed Description	64
7.14.2 Constructor & Destructor Documentation	64
7.14.2.1 SensorState()	64
7.14.3 Member Data Documentation	64
7.14.3.1 consecutive_violations	64
7.14.3.2 has_previous	64
7.14.3.3 previous_value	64
7.15 astralog::SensorRuleSet::StatefulEntry Struct Reference	65
7.15.1 Member Data Documentation	65
7.15.1.1 rule	65
7.15.1.2 slot	66
7.16 astralog::TimestampGroup Struct Reference	66
7.16.1 Detailed Description	66
7.16.2 Member Data Documentation	67
7.16.2.1 readings	67
7.16.2.2 timestamp	67
7.17 astralog::TokenViewEntry Struct Reference	67
7.17.1 Member Data Documentation	67
7.17.1.1 sensor_id	67
7.17.1.2 token	67

8 File Documentation	69
8.1 src/batch_accumulator.hpp File Reference	69
8.1.1 Detailed Description	70
8.1.1.1 Design Notes	70
8.2 batch_accumulator.hpp	71
8.3 src/csv_parser.hpp File Reference	73
8.3.1 Detailed Description	75
8.3.2 Macro Definition Documentation	75
8.3.2.1 MAP_POPULATE	75
8.4 csv_parser.hpp	76
8.5 src/main.cpp File Reference	79
8.5.1 Detailed Description	80
8.5.2 Function Documentation	80
8.5.2.1 main()	80
8.6 src/output_formatter.hpp File Reference	81
8.6.1 Detailed Description	83
8.7 output_formatter.hpp	83
8.8 src/rules_engine.hpp File Reference	84
8.8.1 Detailed Description	86
8.9 rules_engine.hpp	86
8.10 src/timestamp_processor.hpp File Reference	90
8.10.1 Detailed Description	92
8.10.1.1 Processing Pipeline	92
8.10.1.2 Parallelism Characteristics	93
8.11 timestamp_processor.hpp	93
8.12 src/types.hpp File Reference	99
8.12.1 Detailed Description	101
8.13 types.hpp	101
8.14 src/yaml_parser.hpp File Reference	104
8.14.1 Detailed Description	105
8.15 yaml_parser.hpp	105
Index	107

Chapter 1

Directory Hierarchy

1.1 Directories

src	9
batch_accumulator.hpp	69
csv_parser.hpp	73
main.cpp	79
output_formatter.hpp	81
rules_engine.hpp	84
timestamp_processor.hpp	90
types.hpp	99
yaml_parser.hpp	104

Chapter 2

Namespace Index

2.1 Namespace List

Here is a list of all namespaces with brief descriptions:

astralog	11
astralog::detail	36

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

astralog::BatchAccumulator	Accumulates valid records and flushes them in configurable batches	37
astralog::BatchConfig	Configuration for the batch accumulator	41
astralog::detail::MiniJsonParser::JsonValue		43
astralog::MappedFile	RAII wrapper for a read-only memory-mapped file	45
astralog::detail::MiniJsonParser		47
astralog::ParsedRecord		48
astralog::PipelineState	Wraps per-sensor SensorState objects so they persist across batch boundaries	50
astralog::PreparedRules	Rule lookup structures built once after configuration loading	51
astralog::ProcessingResult	Aggregated output of the full processing pipeline	53
astralog::TimestampGroup::Reading		55
astralog::RuleDefinition	In-memory representation of a single monitoring rule loaded from JSON	56
astralog::RuleViolation	Represents a single rule violation at a given timestamp	59
astralog::SensorRuleSet	Pre-built lookup table mapping a sensor token to its applicable rules, sorted by priority within each type	62
astralog::SensorState	Per-sensor mutable state for sequential rule evaluation	63
astralog::SensorRuleSet::StatefulEntry		65
astralog::TimestampGroup	Groups all valid sensor readings that share the same timestamp	66
astralog::TokenViewEntry		67

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

src/ batch_accumulator.hpp	
Batch Accumulator component for AstraLog-HPC	69
src/ csv_parser.hpp	
Memory-mapped CSV parser with parallel line parsing	73
src/ main.cpp	
AstraLog-HPC — Entry point and orchestration	79
src/ output_formatter.hpp	
Spec-compliant output formatting for valid_data.csv and alarms.log	81
src/ rules_engine.hpp	
Rule loading from JSON and inline rule evaluation functions	84
src/ timestamp_processor.hpp	
Per-timestamp grouping, parallel rule evaluation, and correlation	90
src/ types.hpp	
Core types, enumerations, and data structures for AstraLog-HPC	99
src/ yaml_parser.hpp	
Lightweight YAML parser for sensor configuration files	104

Chapter 5

Directory Documentation

5.1 src Directory Reference

Files

- file [batch_accumulator.hpp](#)
Batch Accumulator component for AstraLog-HPC.
- file [csv_parser.hpp](#)
Memory-mapped CSV parser with parallel line parsing.
- file [main.cpp](#)
AstraLog-HPC — Entry point and orchestration.
- file [output_formatter.hpp](#)
Spec-compliant output formatting for valid_data.csv and alarms.log.
- file [rules_engine.hpp](#)
Rule loading from JSON and inline rule evaluation functions.
- file [timestamp_processor.hpp](#)
Per-timestamp grouping, parallel rule evaluation, and correlation.
- file [types.hpp](#)
Core types, enumerations, and data structures for AstraLog-HPC.
- file [yaml_parser.hpp](#)
Lightweight YAML parser for sensor configuration files.

Chapter 6

Namespace Documentation

6.1 astralog Namespace Reference

Namespaces

- namespace [detail](#)

Classes

- class [BatchAccumulator](#)
Accumulates valid records and flushes them in configurable batches.
- struct [BatchConfig](#)
Configuration for the batch accumulator.
- struct [MappedFile](#)
RAII wrapper for a read-only memory-mapped file.
- struct [ParsedRecord](#)
- struct [PipelineState](#)
Wraps per-sensor [SensorState](#) objects so they persist across batch boundaries.
- struct [PreparedRules](#)
Rule lookup structures built once after configuration loading.
- struct [ProcessingResult](#)
Aggregated output of the full processing pipeline.
- struct [RuleDefinition](#)
In-memory representation of a single monitoring rule loaded from JSON.
- struct [RuleViolation](#)
Represents a single rule violation at a given timestamp.
- struct [SensorRuleSet](#)
Pre-built lookup table mapping a sensor token to its applicable rules, sorted by priority within each type.
- struct [SensorState](#)
Per-sensor mutable state for sequential rule evaluation.
- struct [TimestampGroup](#)
Groups all valid sensor readings that share the same timestamp.
- struct [TokenViewEntry](#)

Typedefs

- using `TokenViewMap` = `std::vector<TokenViewEntry>`

Enumerations

- enum class `BatchStrategy` : `uint8_t` { `COUNT` = 0 , `TIME` = 1 }
Batch flushing strategy.
- enum class `Priority` : `uint8_t` { `LOW` = 0 , `MEDIUM` = 1 , `HIGH` = 2 }
Alert priority levels (LOW < MEDIUM < HIGH).
- enum class `RuleType` : `uint8_t` { `THRESHOLD` = 0 , `STEP_DIFF` = 1 , `STATEFUL` = 2 , `CORRELATION` = 3 }
Monitoring rule types.
- enum class `CompOp` : `uint8_t` {
 `GT` , `GE` , `LT` , `LE` ,
 `EQ` , `NE` }
Comparison operators for rule evaluation.

Functions

- `const char * strategy_to_string (BatchStrategy s)` noexcept
- `BatchStrategy string_to_strategy (const std::string &s)`
- `std::vector< std::string > build_inverse_token_map (const std::unordered_map< std::string, int > &token_map)`
Build an inverse token map: sensor_token (int) → sensor_id (string).
- `TokenViewMap build_token_view_map (const std::vector< std::string > &inverse_token_map)`
Build a string_view token lookup table backed by inverse_token_map storage.
- `int lookup_sensor_token (const TokenViewMap &token_map, std::string_view sensor_id)` noexcept
- `std::vector< size_t > build_line_offsets (const char *data, size_t size)`
Build a vector of byte offsets for each data line (skipping header).
- `std::string_view get_line (const char *data, size_t size, size_t offset)` noexcept
Extract a line as string_view from mmap data at the given offset.
- `bool tokenize_csv_line (std::string_view line, std::string_view &f0, std::string_view &f1, std::string_view &f2, std::string_view &f3)` noexcept
Tokenize a CSV line into exactly 4 comma-separated fields.
- `bool validate_schema (std::string_view f0, std::string_view f1, std::string_view f2, std::string_view f3)` noexcept
ESA-compliant schema validation.
- `bool parse_value (std::string_view field, double &val)` noexcept
Parse a value field to double using std::from_chars (C++17). Non-allocating, exception-free.
- `std::vector< ParsedRecord > parse_csv_parallel (const MappedFile &csv_file, const std::vector< size_t > &line_offsets, const TokenViewMap &token_map, uint64_t &total_valid, uint64_t &total_invalid)`
Parse the entire CSV file in parallel, returning validated records in original CSV line order.
- `std::string format_value (double val)`
Format a double to string, stripping trailing zeros after the decimal point but keeping at least one decimal digit.
- `std::string format_nominal_line (const TimestampGroup &group, const std::vector< std::string > &inverse_token_map)`
Format a NOMINAL line for valid_data.csv.
- `std::string format_alarm_lines (const std::string ×tamp, const std::vector< RuleViolation > &violations, const std::vector< std::string > &inverse_token_map)`
Format alarm lines for alarms.log.
- `std::vector< RuleDefinition > load_rules (const std::string &filepath)`

- Load monitoring rules from a JSON file.*

 - void [sort_by_priority](#) (std::vector< const [RuleDefinition](#) * > &rules)

Sort a vector of rule pointers by priority (HIGH first, then MEDIUM, then LOW).
 - void [assign_sensor_tokens](#) (std::vector< [RuleDefinition](#) > &rules, const std::unordered_map< std::string, int > &token_map)

Assign sensor_token_idx to each non-correlation rule based on the provided token map.
 - bool [evaluate_threshold](#) (double value, const [RuleDefinition](#) &rule) noexcept

Evaluate a THRESHOLD (simple) rule: value <op> threshold.
 - bool [evaluate_step_diff](#) (double value, const [SensorState](#) &state, const [RuleDefinition](#) &rule) noexcept

Evaluate a STEP_DIFF rule using SIGNED delta.
 - bool [evaluate_stateful](#) (double value, [SensorState](#) &state, const [RuleDefinition](#) &rule, int rule_slot) noexcept

Evaluate a STATEFUL rule: consecutive violation persistence.
 - [PreparedRules](#) [prepare_rules](#) (std::vector< [RuleDefinition](#) > &all_rules, const std::unordered_map< std::string, int > &token_map, int num_sensors)

Precompute all per-sensor and correlation rule lookup structures.
 - std::vector< [TimestampGroup](#) > [build_timestamp_groups](#) (const std::vector< [ParsedRecord](#) > &records)

Group consecutive records by timestamp.
 - std::vector< std::vector< std::pair< size_t, size_t > > > > [build_per_sensor_sequences](#) (const std::vector< [TimestampGroup](#) > &groups, int num_sensors)

For each sensor, build an ordered sequence of (group_idx, reading_idx) pairs.
 - std::vector< std::vector< [RuleViolation](#) > > [evaluate_rules_parallel](#) (const std::vector< [TimestampGroup](#) > &groups, const std::vector< std::vector< std::pair< size_t, size_t > > > &sensor_sequences, const std::vector< [SensorRuleSet](#) > &sensor_rules, int num_sensors, [PipelineState](#) *pipeline_state=nullptr)

Evaluate threshold, step_diff, and stateful rules for all sensors in parallel using OpenMP.
 - void [evaluate_correlations](#) (const std::vector< [TimestampGroup](#) > &groups, std::vector< std::vector< [RuleViolation](#) > > ×tamp_violations, const std::vector< const [RuleDefinition](#) * > &correlation_rules)

Evaluate correlation rules across all timestamps.
 - void [sort_violations_for_output](#) (std::vector< std::vector< [RuleViolation](#) > > ×tamp_violations)
 - void [generate_output_parallel](#) (const std::vector< [TimestampGroup](#) > &groups, const std::vector< std::vector< [RuleViolation](#) > > ×tamp_violations, const std::vector< std::string > &inverse_token_map, std::vector< std::string > &valid_lines, std::vector< std::string > &alarm_lines)

Generate output strings for all timestamps in parallel.
 - [ProcessingResult](#) [process_pipeline](#) (const std::vector< [ParsedRecord](#) > &records, const [PreparedRules](#) &prepared_rules, const std::vector< std::string > &inverse_token_map, [PipelineState](#) &state, bool verbose=true)

Run the full processing pipeline: group → evaluate → correlate → output.
 - const char * [priority_to_string](#) ([Priority](#) p) noexcept
 - [Priority](#) [string_to_priority](#) (std::string_view s)
 - const char * [ruletype_to_string](#) ([RuleType](#) t) noexcept
 - [RuleType](#) [string_to_ruletype](#) (std::string_view s)
 - [CompOp](#) [string_to_compop](#) (std::string_view s)
 - bool [evaluate_comparison](#) (double lhs, [CompOp](#) op, double rhs) noexcept

Inline comparison: lhs <op> rhs.
 - std::vector< std::string > [load_sensors_from_yaml](#) (const std::string &filepath)

Parse a YAML sensor configuration file and extract all sensor IDs.

6.1.1 Typedef Documentation

6.1.1.1 TokenViewMap

```
using astralog::TokenViewMap = std::vector<TokenViewEntry>
```

6.1.2 Enumeration Type Documentation

6.1.2.1 BatchStrategy

```
enum class astralog::BatchStrategy : uint8_t [strong]
```

Batch flushing strategy.

Enumerator

COUNT	Flush every N valid records.
TIME	Flush every N milliseconds of wall-clock time.

6.1.2.2 CompOp

```
enum class astralog::CompOp : uint8_t [strong]
```

Comparison operators for rule evaluation.

Enumerator

GT	
GE	
LT	
LE	
EQ	
NE	

6.1.2.3 Priority

```
enum class astralog::Priority : uint8_t [strong]
```

Alert priority levels (LOW < MEDIUM < HIGH).

Enumerator

LOW	
MEDIUM	
HIGH	

6.1.2.4 RuleType

```
enum class astralog::RuleType : uint8_t [strong]
```

Monitoring rule types.

Enumerator

THRESHOLD	Simple absolute threshold check ("simple" in JSON).
-----------	---

STEP_DIFF	Signed difference between consecutive readings.
STATEFUL	Consecutive violation persistence counter.
CORRELATION	Logical AND/OR across sub-rules at same timestamp.

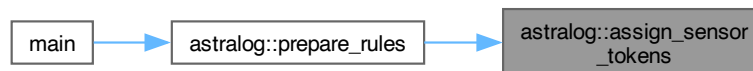
6.1.3 Function Documentation

6.1.3.1 assign_sensor_tokens()

```
void astralog::assign_sensor_tokens (
    std::vector< RuleDefinition > & rules,
    const std::unordered_map< std::string, int > & token_map) [inline]
```

Assign sensor_token_idx to each non-correlation rule based on the provided token map.

Here is the caller graph for this function:



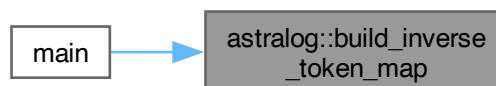
6.1.3.2 build_inverse_token_map()

```
std::vector< std::string > astralog::build_inverse_token_map (
    const std::unordered_map< std::string, int > & token_map) [inline]
```

Build an inverse token map: sensor_token (int) → sensor_id (string).

Given the forward map (sensor_id → token), returns a vector where index i contains the sensor name for token i.

Here is the caller graph for this function:

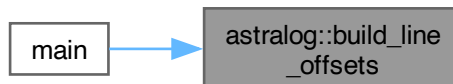


6.1.3.3 build_line_offsets()

```
std::vector< size_t > astralog::build_line_offsets (
    const char * data,
    size_t size) [inline]
```

Build a vector of byte offsets for each data line (skipping header).

Enables O(1) random access to any CSV data line by index. Here is the caller graph for this function:



6.1.3.4 build_per_sensor_sequences()

```
std::vector< std::vector< std::pair< size_t, size_t > > > astralog::build_per_sensor_↵
sequences (
    const std::vector< TimestampGroup > & groups,
    int num_sensors) [inline]
```

For each sensor, build an ordered sequence of (group_idx, reading_idx) pairs.

This allows Phase 3 to iterate a single sensor's readings in temporal order without scanning all records.

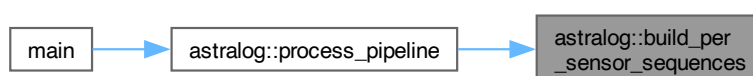
Parameters

<i>groups</i>	Timestamp groups from Phase 1
<i>num_sensors</i>	Total number of known sensors

Returns

`sensor_sequences[sensor_token]` = ordered list of (group_idx, reading_idx)

Here is the caller graph for this function:



6.1.3.5 build_timestamp_groups()

```
std::vector< TimestampGroup > astralog::build_timestamp_groups (
    const std::vector< ParsedRecord > & records) [inline]
```

Group consecutive records by timestamp.

Assumes records are in CSV order (timestamps are monotonically ordered). Records with unknown sensor_token (-1) are still included for output completeness but won't have rules evaluated against them.

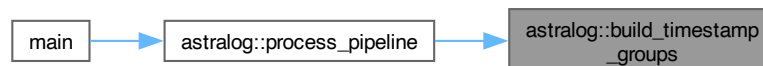
Parameters

<i>records</i>	Parsed and validated CSV records in order
----------------	---

Returns

Vector of [TimestampGroup](#), one per unique timestamp

Here is the caller graph for this function:



6.1.3.6 build_token_view_map()

```
TokenViewMap astralog::build_token_view_map (
    const std::vector< std::string > & inverse_token_map) [inline]
```

Build a string_view token lookup table backed by inverse_token_map storage.

The returned keys are non-owning views into inverse_token_map, so the inverse_token_map vector must outlive the returned lookup table. Here is the caller graph for this function:

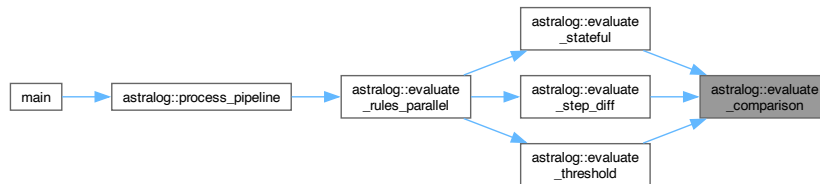


6.1.3.7 evaluate_comparison()

```
bool astralog::evaluate_comparison (
    double lhs,
    CompOp op,
    double rhs) [inline], [noexcept]
```

Inline comparison: lhs <op> rhs.

Here is the caller graph for this function:



6.1.3.8 evaluate_correlations()

```
void astralog::evaluate_correlations (
    const std::vector< TimestampGroup > & groups,
    std::vector< std::vector< RuleViolation > > & timestamp_violations,
    const std::vector< const RuleDefinition * > & correlation_rules) [inline]
```

Evaluate correlation rules across all timestamps.

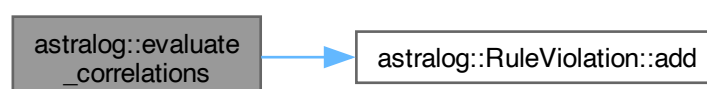
For each timestamp, checks if the sub-rules referenced by each correlation rule have fired. If the logical condition (AND/OR) is satisfied, appends a correlation violation.

Must run after Phase 3 so that all sub-rule results are available.

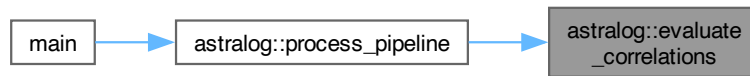
Parameters

<i>groups</i>	Timestamp groups
<i>timestamp_violations</i>	Per-timestamp violations (modified in-place)
<i>correlation_rules</i>	All correlation-type rules (sorted by priority)

Here is the call graph for this function:



Here is the caller graph for this function:



6.1.3.9 evaluate_rules_parallel()

```

std::vector< std::vector< RuleViolation > > astralog::evaluate_rules_parallel (
    const std::vector< TimestampGroup > & groups,
    const std::vector< std::vector< std::pair< size_t, size_t > > > & sensor_sequences,
    const std::vector< SensorRuleSet > & sensor_rules,
    int num_sensors,
    PipelineState * pipeline_state = nullptr) [inline]
  
```

Evaluate threshold, step_diff, and stateful rules for all sensors in parallel using OpenMP.

Each sensor is processed by a single thread, ensuring correct sequential ordering for step_diff (which needs the previous reading) and stateful (which needs consecutive violation counts). Different sensors are fully independent, providing parallelism proportional to the number of sensors.

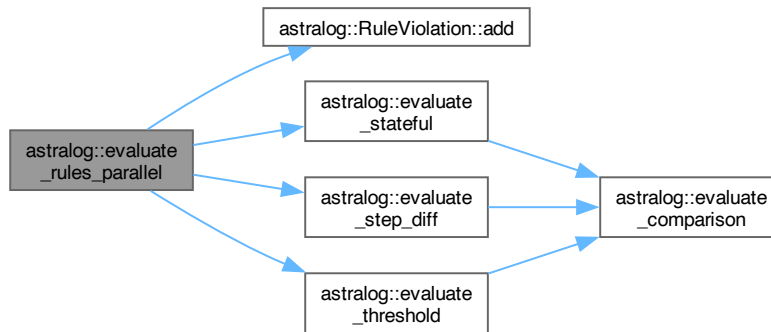
Parameters

<i>groups</i>	Timestamp groups
<i>sensor_sequences</i>	Per-sensor reading sequences from Phase 2
<i>sensor_rules</i>	Per-sensor rule lookup tables
<i>num_sensors</i>	Total number of known sensors
<i>pipeline_state</i>	Optional pointer to PipelineState for cross-batch sensor state persistence. If non-null, sensor states are read from and written back to this object.

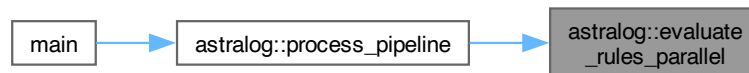
Returns

Per-timestamp violation map: timestamp_idx → violations

Here is the call graph for this function:



Here is the caller graph for this function:

**6.1.3.10 evaluate_stateful()**

```

bool astralog::evaluate_stateful (
    double value,
    SensorState & state,
    const RuleDefinition & rule,
    int rule_slot) [inline], [noexcept]
  
```

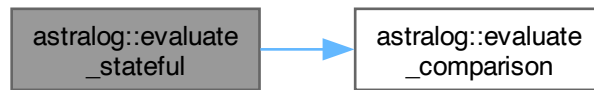
Evaluate a STATEFUL rule: consecutive violation persistence.

Maintains a per-(sensor, rule) counter. The alarm is triggered when the counter reaches consecutive_↔ measurements, and on every subsequent continuous violation.

Parameters

<i>rule_slot</i>	Index into state.consecutive_violations[] for this rule
------------------	---

Here is the call graph for this function:



Here is the caller graph for this function:



6.1.3.11 evaluate_step_diff()

```

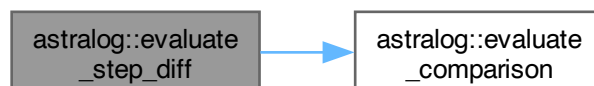
bool astralog::evaluate_step_diff (
    double value,
    const SensorState & state,
    const RuleDefinition & rule) [inline], [noexcept]
  
```

Evaluate a STEP_DIFF rule using SIGNED delta.

$\text{delta} = \text{current_value} - \text{previous_value}$

The spec defines `step_difference` as checking (current - previous) against the operator and threshold. For example: operator "<", value -2.0 $\rightarrow$ fires when $\text{delta} < -2.0$ (i.e. a drop of more than 2 units)

NOTE: No `std::abs()` — the signed delta is compared directly. This is critical for correctness: `abs()` would invert the sign and cause operators like "<" with negative thresholds to never trigger. Here is the call graph for this function:



Here is the caller graph for this function:



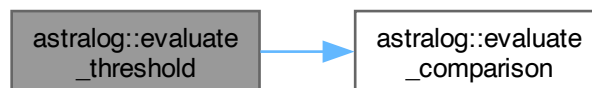
6.1.3.12 evaluate_threshold()

```

bool astralog::evaluate_threshold (
    double value,
    const RuleDefinition & rule) [inline], [noexcept]
  
```

Evaluate a THRESHOLD (simple) rule: value <op> threshold.

Here is the call graph for this function:



Here is the caller graph for this function:



6.1.3.13 format_alarm_lines()

```

std::string astralog::format_alarm_lines (
    const std::string & timestamp,
    const std::vector< RuleViolation > & violations,
    const std::vector< std::string > & inverse_token_map) [inline]
  
```

Format alarm lines for alarms.log.

For simple/step_diff/stateful rules (single sensor): TIMESTAMP;RULE_ID;PRIORITY;SENSOR;VALUE

For correlation rules (multiple sensors): TIMESTAMP;RULE_ID;PRIORITY;SENSOR_1,SENSOR_2;VALUE_1,VALUE_2

Parameters

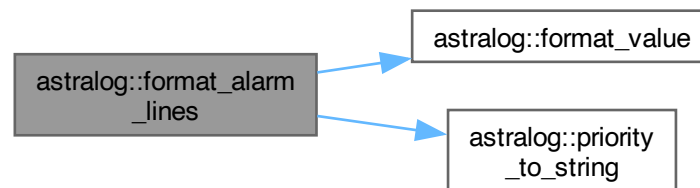
<i>timestamp</i>	The timestamp string
------------------	----------------------

<i>violations</i>	All rule violations at this timestamp
<i>inverse_token_map</i>	Lookup: sensor_token → sensor_id string

Returns

Concatenated alarm lines (each with trailing newline)

Here is the call graph for this function:



Here is the caller graph for this function:

**6.1.3.14 format_nominal_line()**

```

std::string astralog::format_nominal_line (
    const TimestampGroup & group,
    const std::vector< std::string > & inverse_token_map) [inline]
  
```

Format a NOMINAL line for valid_data.csv.

Format: `TIMESTAMP;NOMINAL;SENSOR_1:VALUE_1|SENSOR_2:VALUE_2|...`

Parameters

<i>group</i>	The timestamp group containing all sensor readings
<i>inverse_token_map</i>	Lookup: sensor_token → sensor_id string

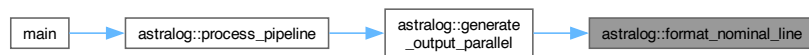
Returns

Formatted line (with trailing newline)

Here is the call graph for this function:



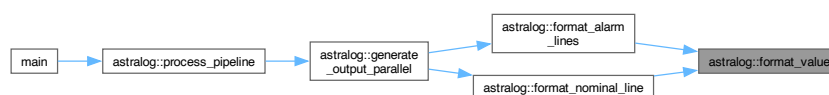
Here is the caller graph for this function:

**6.1.3.15 format_value()**

```
std::string astralog::format_value (
    double val) [inline]
```

Format a double to string, stripping trailing zeros after the decimal point but keeping at least one decimal digit.

Examples: 35.979 → "35.979", 36.0 → "36.0", 100.0 → "100.0" Here is the caller graph for this function:

**6.1.3.16 generate_output_parallel()**

```
void astralog::generate_output_parallel (
    const std::vector< TimestampGroup > & groups,
    const std::vector< std::vector< RuleViolation > > & timestamp_violations,
    const std::vector< std::string > & inverse_token_map,
    std::vector< std::string > & valid_lines,
    std::vector< std::string > & alarm_lines) [inline]
```

Generate output strings for all timestamps in parallel.

For each timestamp:

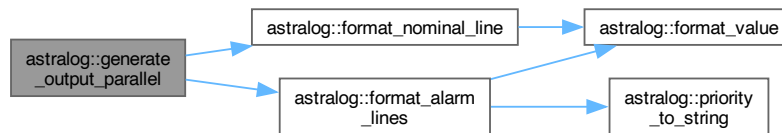
- If no violations → NOMINAL line (written to valid_data.csv)
- If any violations → alarm lines (written to alarms.log)

Parameters

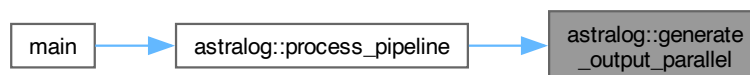
	<i>groups</i>	Timestamp groups
--	---------------	------------------

	<i>timestamp_violations</i>	Per-timestamp violations
out	<i>valid_lines</i>	NOMINAL output per timestamp (empty if anomalous)
out	<i>alarm_lines</i>	Alarm output per timestamp (empty if nominal)

Here is the call graph for this function:



Here is the caller graph for this function:



6.1.3.17 get_line()

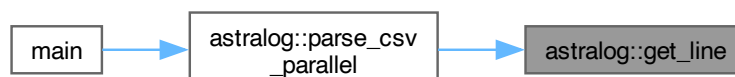
```

std::string_view astralog::get_line (
    const char * data,
    size_t size,
    size_t offset) [inline], [noexcept]

```

Extract a line as `string_view` from mmap data at the given offset.

Here is the caller graph for this function:



6.1.3.18 load_rules()

```
std::vector< RuleDefinition > astralog::load_rules (
    const std::string & filepath) [inline]
```

Load monitoring rules from a JSON file.

Expects a top-level JSON array of rule objects. Each object must have at least "rule_id" and "type" fields. [Priority](#) defaults to LOW if absent.

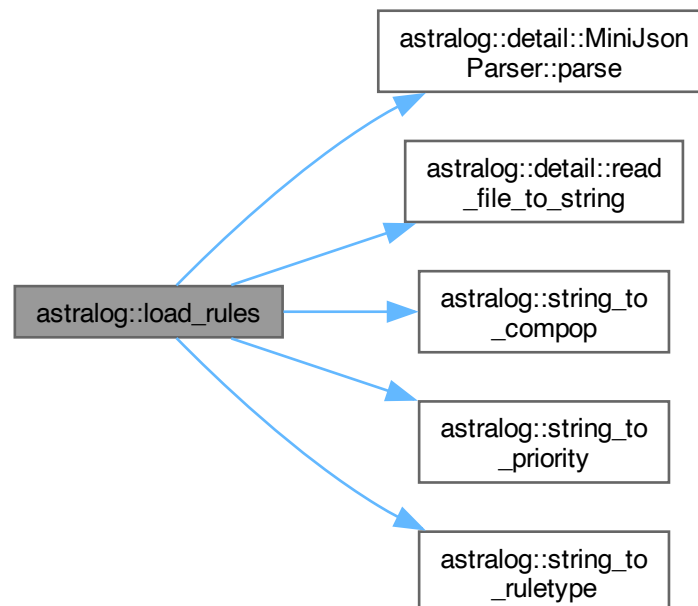
Parameters

<i>filepath</i>	Path to rules.json
-----------------	--------------------

Returns

Vector of [RuleDefinition](#) (unsorted)

Here is the call graph for this function:



Here is the caller graph for this function:



6.1.3.19 load_sensors_from_yaml()

```
std::vector< std::string > astralog::load_sensors_from_yaml (
    const std::string & filepath) [inline]
```

Parse a YAML sensor configuration file and extract all sensor IDs.

Parameters

<i>filepath</i>	Path to the sensors.yaml file
-----------------	-------------------------------

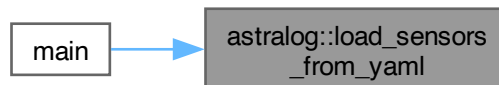
Returns

Vector of sensor ID strings (e.g. "TEMP-001", "PRES-002")

Exceptions

<i>std::runtime_error</i>	if the file cannot be opened
---------------------------	------------------------------

Here is the caller graph for this function:



6.1.3.20 lookup_sensor_token()

```
int astralog::lookup_sensor_token (
    const TokenViewMap & token_map,
    std::string_view sensor_id) [inline], [noexcept]
```

Here is the caller graph for this function:



6.1.3.21 parse_csv_parallel()

```
std::vector< ParsedRecord > astralog::parse_csv_parallel (
    const MappedFile & csv_file,
    const std::vector< size_t > & line_offsets,
    const TokenViewMap & token_map,
    uint64_t & total_valid,
    uint64_t & total_invalid) [inline]
```

Parse the entire CSV file in parallel, returning validated records in original CSV line order.

Uses OpenMP static scheduling to ensure each thread processes a contiguous block of lines. Concatenating thread-local results in thread order preserves the original file ordering.

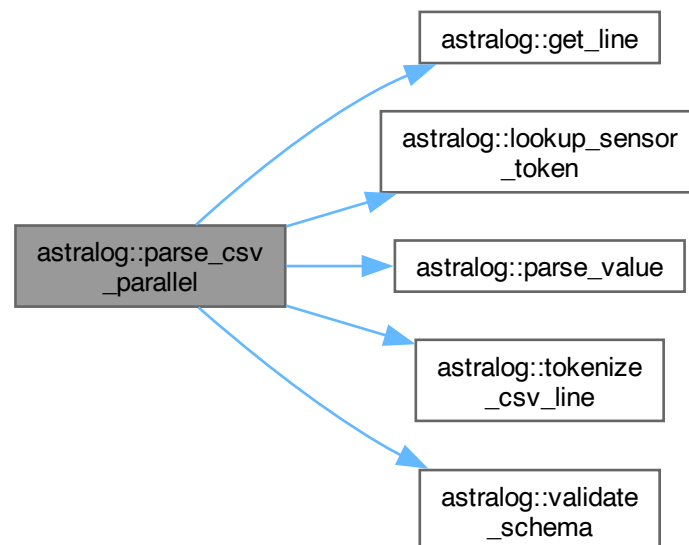
Parameters

	<i>csv_file</i>	Memory-mapped CSV file
	<i>line_offsets</i>	Pre-computed line offset index
	<i>token_map</i>	sensor_id → integer token mapping
out	<i>total_valid</i>	Count of valid rows
out	<i>total_invalid</i>	Count of invalid/dropped rows

Returns

Vector of [ParsedRecord](#) in original CSV order

Here is the call graph for this function:



Here is the caller graph for this function:



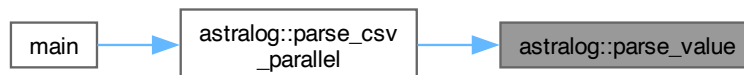
6.1.3.22 parse_value()

```

bool astralog::parse_value (
    std::string_view field,
    double & val) [inline], [noexcept]
  
```

Parse a value field to double using `std::from_chars` (C++17). Non-allocating, exception-free.

Here is the caller graph for this function:



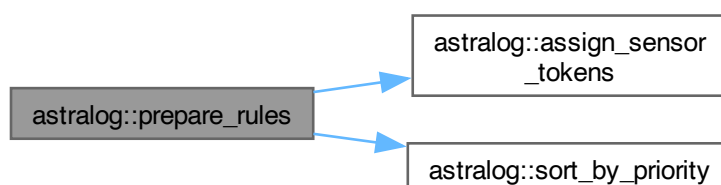
6.1.3.23 prepare_rules()

```

PreparedRules astralog::prepare_rules (
    std::vector< RuleDefinition > & all_rules,
    const std::unordered_map< std::string, int > & token_map,
    int num_sensors) [inline]
  
```

Precompute all per-sensor and correlation rule lookup structures.

This removes repeated rule categorisation, priority sorting, and sensor-rule table construction from the per-batch processing path. Here is the call graph for this function:



Here is the caller graph for this function:

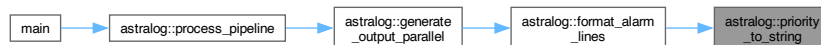


6.1.3.24 priority_to_string()

```

const char * astralog::priority_to_string (
    Priority p) [inline], [noexcept]
  
```

Here is the caller graph for this function:



6.1.3.25 process_pipeline()

```

ProcessingResult astralog::process_pipeline (
    const std::vector< ParsedRecord > & records,
    const PreparedRules & prepared_rules,
    const std::vector< std::string > & inverse_token_map,
    PipelineState & state,
    bool verbose = true) [inline]
  
```

Run the full processing pipeline: group → evaluate → correlate → output.

This overload accepts a [PipelineState](#) reference for cross-batch sensor state persistence. When processing multiple batches, pass the same [PipelineState](#) object to each call so that step-diff previous_values and stateful consecutive↔_violations counters carry forward.

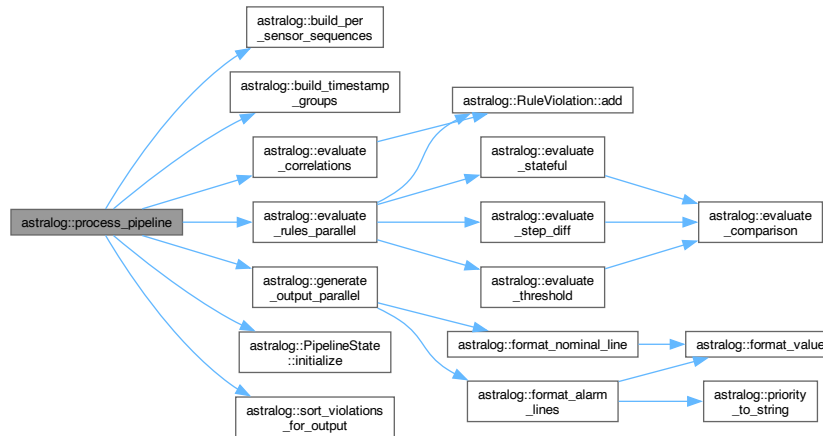
Parameters

<i>records</i>	Parsed and validated CSV records in order
<i>prepared_rules</i>	Precomputed per-sensor and correlation rules
<i>inverse_token_map</i>	sensor_token to sensor_id lookup
<i>state</i>	PipelineState for cross-batch persistence
<i>verbose</i>	Emit per-phase timing logs when true

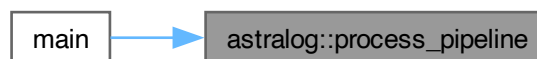
Returns

[ProcessingResult](#) with output strings and stats

Here is the call graph for this function:



Here is the caller graph for this function:



6.1.3.26 ruletype_to_string()

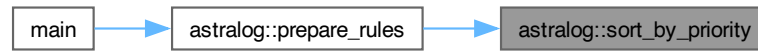
```
const char * astralog::ruletype_to_string (
    RuleType t) [inline], [noexcept]
```

6.1.3.27 sort_by_priority()

```
void astralog::sort_by_priority (
    std::vector< const RuleDefinition * > & rules) [inline]
```

Sort a vector of rule pointers by priority (HIGH first, then MEDIUM, then LOW).

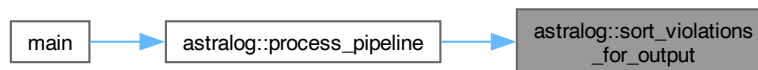
The spec requires: "rules with higher priority are evaluated before the others" within each rule type. Here is the caller graph for this function:



6.1.3.28 sort_violations_for_output()

```
void astralog::sort_violations_for_output (
    std::vector< std::vector< RuleViolation > > & timestamp_violations) [inline]
```

Here is the caller graph for this function:



6.1.3.29 strategy_to_string()

```
const char * astralog::strategy_to_string (
    BatchStrategy s) [inline], [noexcept]
```

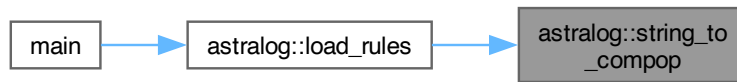
Here is the caller graph for this function:



6.1.3.30 string_to_compop()

```
CompOp astralog::string_to_compop (  
    std::string_view s) [inline]
```

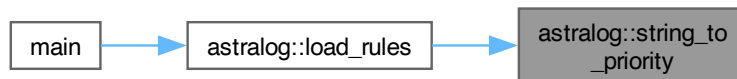
Here is the caller graph for this function:



6.1.3.31 string_to_priority()

```
Priority astralog::string_to_priority (  
    std::string_view s) [inline]
```

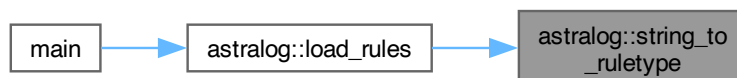
Here is the caller graph for this function:



6.1.3.32 string_to_ruletype()

```
RuleType astralog::string_to_ruletype (  
    std::string_view s) [inline]
```

Here is the caller graph for this function:



6.1.3.33 string_to_strategy()

```
BatchStrategy astralog::string_to_strategy (  
    const std::string & s) [inline]
```

Here is the caller graph for this function:



6.1.3.34 tokenize_csv_line()

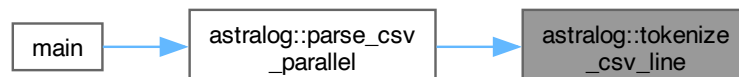
```
bool astralog::tokenize_csv_line (  
    std::string_view line,  
    std::string_view & f0,  
    std::string_view & f1,  
    std::string_view & f2,  
    std::string_view & f3) [inline], [noexcept]
```

Tokenize a CSV line into exactly 4 comma-separated fields.

Returns

true if exactly 4 fields found (no more, no fewer)

Here is the caller graph for this function:

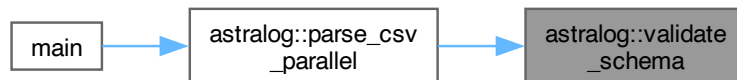


6.1.3.35 validate_schema()

```
bool astralog::validate_schema (
    std::string_view f0,
    std::string_view f1,
    std::string_view f2,
    std::string_view f3) [inline], [noexcept]
```

ESA-compliant schema validation.

Rejects: empty fields, "ERR" or "CORRUPT" markers in any field. Here is the caller graph for this function:



6.2 astralog::detail Namespace Reference

Classes

- class [MiniJsonParser](#)

Functions

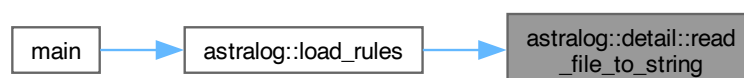
- std::string [read_file_to_string](#) (const std::string &filepath)

6.2.1 Function Documentation

6.2.1.1 read_file_to_string()

```
std::string astralog::detail::read_file_to_string (
    const std::string & filepath) [inline]
```

Here is the caller graph for this function:



Chapter 7

Class Documentation

7.1 astralog::BatchAccumulator Class Reference

Accumulates valid records and flushes them in configurable batches.

```
#include <batch_accumulator.hpp>
```

Collaboration diagram for astralog::BatchAccumulator:

astralog::BatchAccumulator	
+	BatchAccumulator()
+	add_record()
+	should_flush()
+	flush()
+	flush_remaining()
+	batch_count()
+	buffered_count()
+	config()

Public Member Functions

- [BatchAccumulator](#) (const [BatchConfig](#) &config, const std::vector< std::string > &inverse_token_map)
- void [add_record](#) ([ParsedRecord](#) rec)
Push one valid record into the accumulator buffer.
- bool [should_flush](#) () const
Check if the flush threshold has been met.
- std::vector< [ParsedRecord](#) > [flush](#) ()

- *Flush the current buffer: return records and write audit file.*
 • `std::vector< ParsedRecord > flush_remaining ()`
Flush any remaining records (final partial batch at end-of-file).
- `int batch_count () const noexcept`
How many batches have been flushed so far.
- `size_t buffered_count () const noexcept`
How many records are currently buffered (not yet flushed).
- `const BatchConfig & config () const noexcept`
Get the batch configuration.

7.1.1 Detailed Description

Accumulates valid records and flushes them in configurable batches.

Usage: `BatchAccumulator acc(config); for (auto& record : all_valid_records) { acc.add_record(std::move(record)); if (acc.should_flush()) { auto batch = acc.flush(); // process batch through rule engine... } } auto final_batch = acc.flush_remaining(); if (!final_batch.empty()) { // process final partial batch... }`

7.1.2 Constructor & Destructor Documentation

7.1.2.1 BatchAccumulator()

```
astralog::BatchAccumulator::BatchAccumulator (
    const BatchConfig & config,
    const std::vector< std::string > & inverse_token_map) [inline], [explicit]
```

Here is the call graph for this function:



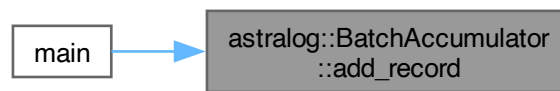
7.1.3 Member Function Documentation

7.1.3.1 add_record()

```
void astralog::BatchAccumulator::add_record (
    ParsedRecord rec) [inline]
```

Push one valid record into the accumulator buffer.

Here is the caller graph for this function:

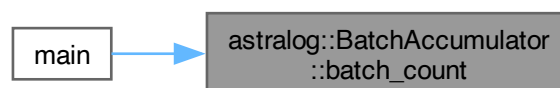


7.1.3.2 batch_count()

```
int astralog::BatchAccumulator::batch_count () const [inline], [noexcept]
```

How many batches have been flushed so far.

Here is the caller graph for this function:



7.1.3.3 buffered_count()

```
size_t astralog::BatchAccumulator::buffered_count () const [inline], [noexcept]
```

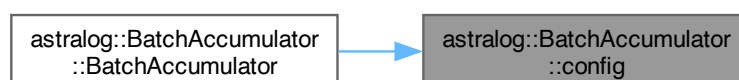
How many records are currently buffered (not yet flushed).

7.1.3.4 config()

```
const BatchConfig & astralog::BatchAccumulator::config () const [inline], [noexcept]
```

Get the batch configuration.

Here is the caller graph for this function:



7.1.3.5 flush()

```
std::vector< ParsedRecord > astralog::BatchAccumulator::flush () [inline]
```

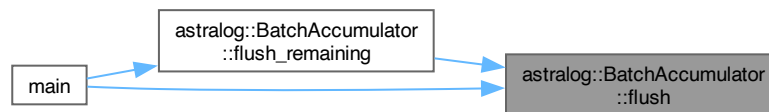
Flush the current buffer: return records and write audit file.

Resets the internal buffer and timer for the next batch.

Returns

Vector of [ParsedRecord](#) that were in the buffer

Here is the caller graph for this function:



7.1.3.6 flush_remaining()

```
std::vector< ParsedRecord > astralog::BatchAccumulator::flush_remaining () [inline]
```

Flush any remaining records (final partial batch at end-of-file).

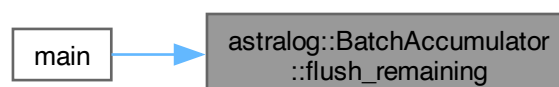
Returns

Vector of remaining [ParsedRecord](#) (may be empty)

Here is the call graph for this function:



Here is the caller graph for this function:

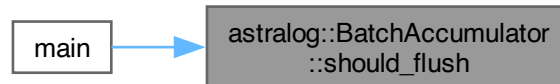


7.1.3.7 should_flush()

```
bool astralog::BatchAccumulator::should_flush () const [inline]
```

Check if the flush threshold has been met.

For COUNT: true when buffer size $\geq$ count_threshold. For TIME: true when wall-clock elapsed $\geq$ time_threshold_ms. Here is the caller graph for this function:



The documentation for this class was generated from the following file:

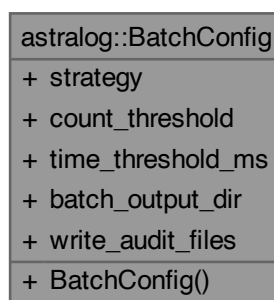
- [src/batch_accumulator.hpp](#)

7.2 astralog::BatchConfig Struct Reference

Configuration for the batch accumulator.

```
#include <batch_accumulator.hpp>
```

Collaboration diagram for astralog::BatchConfig:



Public Member Functions

- [BatchConfig\(\)](#)

Public Attributes

- [BatchStrategy strategy](#)
COUNT or TIME.
- `size_t` [count_threshold](#)
For COUNT: flush every N valid records.
- `double` [time_threshold_ms](#)
For TIME: flush every N ms.
- `std::string` [batch_output_dir](#)
Where to write .txt batch audit files.
- `bool` [write_audit_files](#)
Disable for clean benchmark timing.

7.2.1 Detailed Description

Configuration for the batch accumulator.

7.2.2 Constructor & Destructor Documentation

7.2.2.1 BatchConfig()

```
astralog::BatchConfig::BatchConfig () [inline]
```

7.2.3 Member Data Documentation

7.2.3.1 batch_output_dir

```
std::string astralog::BatchConfig::batch_output_dir
```

Where to write .txt batch audit files.

7.2.3.2 count_threshold

```
size_t astralog::BatchConfig::count_threshold
```

For COUNT: flush every N valid records.

7.2.3.3 strategy

```
BatchStrategy astralog::BatchConfig::strategy
```

COUNT or TIME.

7.2.3.4 time_threshold_ms

```
double astralog::BatchConfig::time_threshold_ms
```

For TIME: flush every N ms.

7.2.3.5 write_audit_files

```
bool astralog::BatchConfig::write_audit_files
```

Disable for clean benchmark timing.

The documentation for this struct was generated from the following file:

- [src/batch_accumulator.hpp](#)

7.3 astralog::detail::MiniJsonParser::JsonValue Struct Reference

```
#include <rules_engine.hpp>
```

Collaboration diagram for astralog::detail::MiniJsonParser::JsonValue:

astralog::detail::MiniJsonParser::JsonValue	
+	type
+	str_val
+	num_val
+	arr_val
+	obj_val
+	find()
+	as_string()
+	as_number()
+	as_int()

Public Types

- enum [Type](#) {
[STRING](#) , [NUMBER](#) , [ARRAY](#) , [OBJECT](#) ,
[NONE](#) }

Public Member Functions

- const [JsonValue](#) * [find](#) (const std::string &key) const
- std::string [as_string](#) (const std::string &def="") const
- double [as_number](#) (double def=0.0) const
- int [as_int](#) (int def=0) const

Public Attributes

- enum [astralog::detail::MiniJsonParser::JsonValue::Type](#) type = NONE
- std::string [str_val](#)
- double [num_val](#) = 0.0
- std::vector< [JsonValue](#) > [arr_val](#)
- std::vector< std::pair< std::string, [JsonValue](#) > > [obj_val](#)

7.3.1 Member Enumeration Documentation

7.3.1.1 Type

```
enum astralog::detail::MiniJsonParser::JsonValue::Type
```

Enumerator

STRING	
NUMBER	
ARRAY	
OBJECT	
NONE	

7.3.2 Member Function Documentation

7.3.2.1 as_int()

```
int astralog::detail::MiniJsonParser::JsonValue::as\_int (
    int def = 0) const [inline]
```

7.3.2.2 as_number()

```
double astralog::detail::MiniJsonParser::JsonValue::as\_number (
    double def = 0.0) const [inline]
```

7.3.2.3 as_string()

```
std::string astralog::detail::MiniJsonParser::JsonValue::as\_string (
    const std::string & def = "") const [inline]
```

7.3.2.4 find()

```
const JsonValue * astralog::detail::MiniJsonParser::JsonValue::find (
    const std::string & key) const [inline]
```

7.3.3 Member Data Documentation

7.3.3.1 arr_val

```
std::vector<JsonValue> astralog::detail::MiniJsonParser::JsonValue::arr_val
```

7.3.3.2 num_val

```
double astralog::detail::MiniJsonParser::JsonValue::num_val = 0.0
```

7.3.3.3 obj_val

```
std::vector<std::pair<std::string, JsonValue> > astralog::detail::MiniJsonParser::JsonValue::obj_val
```

7.3.3.4 str_val

```
std::string astralog::detail::MiniJsonParser::JsonValue::str_val
```

7.3.3.5 type

```
enum astralog::detail::MiniJsonParser::JsonValue::Type astralog::detail::MiniJsonParser::JsonValue::type = NONE
```

The documentation for this struct was generated from the following file:

- [src/rules_engine.hpp](#)

7.4 astralog::MappedFile Struct Reference

RAII wrapper for a read-only memory-mapped file.

```
#include <csv_parser.hpp>
```

Collaboration diagram for astralog::MappedFile:

astralog::MappedFile
+ data
+ size
+ fd
+ ~MappedFile()
+ open()

Public Member Functions

- [~MappedFile](#) ()
- [bool open](#) (const char *path)
Open and memory-map a file. Returns false on failure.

Public Attributes

- const char * [data](#) = nullptr
- size_t [size](#) = 0
- int [fd](#) = -1

7.4.1 Detailed Description

RAII wrapper for a read-only memory-mapped file.

7.4.2 Constructor & Destructor Documentation

7.4.2.1 ~MappedFile()

```
astralog::MappedFile::~MappedFile () [inline]
```

7.4.3 Member Function Documentation

7.4.3.1 open()

```
bool astralog::MappedFile::open (
    const char * path) [inline]
```

Open and memory-map a file. Returns false on failure.

Here is the call graph for this function:



Here is the caller graph for this function:



7.4.4 Member Data Documentation

7.4.4.1 data

```
const char* astralog::MappedFile::data = nullptr
```

7.4.4.2 fd

```
int astralog::MappedFile::fd = -1
```

7.4.4.3 size

```
size_t astralog::MappedFile::size = 0
```

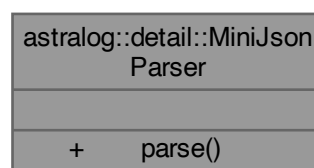
The documentation for this struct was generated from the following file:

- [src/csv_parser.hpp](#)

7.5 astralog::detail::MiniJsonParser Class Reference

```
#include <rules_engine.hpp>
```

Collaboration diagram for astralog::detail::MiniJsonParser:



Classes

- struct [JsonValue](#)

Static Public Member Functions

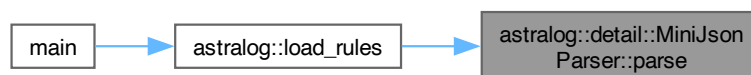
- static [JsonValue parse](#) (const std::string &input)

7.5.1 Member Function Documentation

7.5.1.1 parse()

```
JsonValue astralog::detail::MiniJsonParser::parse (
    const std::string & input) [inline], [static]
```

Here is the caller graph for this function:



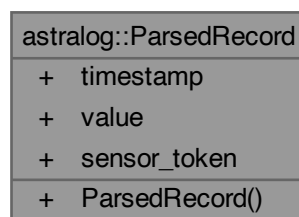
The documentation for this class was generated from the following file:

- src/[rules_engine.hpp](#)

7.6 astralog::ParsedRecord Struct Reference

```
#include <types.hpp>
```

Collaboration diagram for astralog::ParsedRecord:



Public Member Functions

- [ParsedRecord\(\)](#)

Public Attributes

- std::string [timestamp](#)
- double [value](#)
- int [sensor_token](#)

Index into sensor arrays, -1 if unknown.

7.6.1 Constructor & Destructor Documentation

7.6.1.1 ParsedRecord()

```
astralog::ParsedRecord::ParsedRecord () [inline]
```

7.6.2 Member Data Documentation

7.6.2.1 sensor_token

```
int astralog::ParsedRecord::sensor_token
```

Index into sensor arrays, -1 if unknown.

7.6.2.2 timestamp

```
std::string astralog::ParsedRecord::timestamp
```

7.6.2.3 value

```
double astralog::ParsedRecord::value
```

The documentation for this struct was generated from the following file:

- [src/types.hpp](#)

7.7 astralog::PipelineState Struct Reference

Wraps per-sensor [SensorState](#) objects so they persist across batch boundaries.

```
#include <timestamp_processor.hpp>
```

Collaboration diagram for `astralog::PipelineState`:

astralog::PipelineState
+ <code>sensor_states</code>
+ <code>initialized</code>
+ <code>initialize()</code>

Public Member Functions

- void `initialize` (int num_sensors, const std::vector< [SensorRuleSet](#) > &sensor_rules)
Initialize sensor states for the given number of sensors and stateful rule slot counts.

Public Attributes

- std::vector< [SensorState](#) > `sensor_states`
Indexed by sensor_token.
- bool `initialized` = false
True after first batch.

7.7.1 Detailed Description

Wraps per-sensor [SensorState](#) objects so they persist across batch boundaries.

When processing data in batches, step-diff rules need the previous reading from the last record of the prior batch, and stateful rules need their consecutive_violations counters to carry forward.

Usage: `PipelineState` state; auto prepared = prepare_rules(rules, token_map, num_sensors); for each batch: result = process_pipeline(batch, prepared, inverse_token_map, state);

7.7.2 Member Function Documentation

7.7.2.1 initialize()

```
void astralog::PipelineState::initialize (
    int num_sensors,
    const std::vector< SensorRuleSet > & sensor_rules) [inline]
```

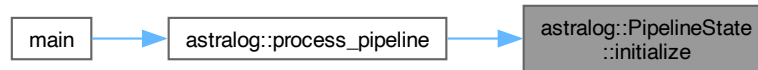
Initialize sensor states for the given number of sensors and stateful rule slot counts.

Parameters

<code>num_sensors</code>	Total number of known sensors
--------------------------	-------------------------------

<code>sensor_rules</code>	Per-sensor rule lookup tables (for stateful slot counts)
---------------------------	--

Here is the caller graph for this function:



7.7.3 Member Data Documentation

7.7.3.1 initialized

```
bool astralog::PipelineState::initialized = false
```

True after first batch.

7.7.3.2 sensor_states

```
std::vector<SensorState> astralog::PipelineState::sensor_states
```

Indexed by sensor_token.

The documentation for this struct was generated from the following file:

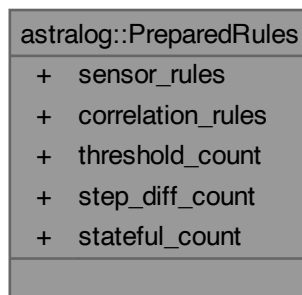
- [src/timestamp_processor.hpp](#)

7.8 astralog::PreparedRules Struct Reference

Rule lookup structures built once after configuration loading.

```
#include <timestamp_processor.hpp>
```

Collaboration diagram for `astralog::PreparedRules`:



Public Attributes

- `std::vector< SensorRuleSet > sensor_rules`
- `std::vector< const RuleDefinition * > correlation_rules`
- `size_t threshold_count = 0`
- `size_t step_diff_count = 0`
- `size_t stateful_count = 0`

7.8.1 Detailed Description

Rule lookup structures built once after configuration loading.

Pointers inside this structure refer to the stable [RuleDefinition](#) objects owned by the rules vector loaded in [main\(\)](#).

7.8.2 Member Data Documentation

7.8.2.1 correlation_rules

```
std::vector<const RuleDefinition*> astralog::PreparedRules::correlation_rules
```

7.8.2.2 sensor_rules

```
std::vector<SensorRuleSet> astralog::PreparedRules::sensor_rules
```

7.8.2.3 stateful_count

```
size_t astralog::PreparedRules::stateful_count = 0
```

7.8.2.4 step_diff_count

```
size_t astralog::PreparedRules::step_diff_count = 0
```

7.8.2.5 threshold_count

```
size_t astralog::PreparedRules::threshold_count = 0
```

The documentation for this struct was generated from the following file:

- [src/timestamp_processor.hpp](#)

7.9 astralog::ProcessingResult Struct Reference

Aggregated output of the full processing pipeline.

```
#include <timestamp_processor.hpp>
```

Collaboration diagram for astralog::ProcessingResult:

astralog::ProcessingResult	
+	valid_lines
+	alarm_lines
+	num_timestamps
+	num_anomalies
+	num_nominal
+	eval_ms
+	total_ms

Public Attributes

- `std::vector< std::string > valid_lines`
NOMINAL lines per timestamp.
- `std::vector< std::string > alarm_lines`
Alarm lines per timestamp.
- `uint64_t num_timestamps`
Total unique timestamps.
- `uint64_t num_anomalies`
Timestamps with 1 violation.
- `uint64_t num_nominal`
Timestamps with 0 violations.
- `double eval_ms`
Phase 3 wall time (ms).
- `double total_ms`
Full pipeline wall time (ms).

7.9.1 Detailed Description

Aggregated output of the full processing pipeline.

7.9.2 Member Data Documentation

7.9.2.1 alarm_lines

```
std::vector<std::string> astralog::ProcessingResult::alarm_lines
```

Alarm lines per timestamp.

7.9.2.2 eval_ms

```
double astralog::ProcessingResult::eval_ms
```

Phase 3 wall time (ms).

7.9.2.3 num_anomalies

```
uint64_t astralog::ProcessingResult::num_anomalies
```

Timestamps with 1 violation.

7.9.2.4 num_nominal

```
uint64_t astralog::ProcessingResult::num_nominal
```

Timestamps with 0 violations.

7.9.2.5 num_timestamps

```
uint64_t astralog::ProcessingResult::num_timestamps
```

Total unique timestamps.

7.9.2.6 total_ms

```
double astralog::ProcessingResult::total_ms
```

Full pipeline wall time (ms).

7.9.2.7 valid_lines

```
std::vector<std::string> astralog::ProcessingResult::valid_lines
```

NOMINAL lines per timestamp.

The documentation for this struct was generated from the following file:

- [src/timestamp_processor.hpp](#)

7.10 astralog::TimestampGroup::Reading Struct Reference

```
#include <types.hpp>
```

Collaboration diagram for astralog::TimestampGroup::Reading:

astralog::TimestampGroup ::Reading	
+	sensor_token
+	value

Public Attributes

- int [sensor_token](#)
- double [value](#)

7.10.1 Member Data Documentation

7.10.1.1 sensor_token

```
int astralog::TimestampGroup::Reading::sensor_token
```

7.10.1.2 value

```
double astralog::TimestampGroup::Reading::value
```

The documentation for this struct was generated from the following file:

- [src/types.hpp](#)

7.11 astralog::RuleDefinition Struct Reference

In-memory representation of a single monitoring rule loaded from JSON.

```
#include <types.hpp>
```

Collaboration diagram for `astralog::RuleDefinition`:

astralog::RuleDefinition
+ rule_id + type + sensor_id + op + threshold + priority + consecutive_measurements + logic + sub_rules + sensor_token_idx + RuleDefinition()

Public Member Functions

- [RuleDefinition](#) ()

Public Attributes

- `std::string` [rule_id](#)
Unique identifier (e.g. "R001").
- [RuleType](#) [type](#)
THRESHOLD, STEP_DIFF, STATEFUL, or CORRELATION.
- `std::string` [sensor_id](#)
Target sensor (empty for correlation).
- [CompOp](#) [op](#)
Comparison operator.
- `double` [threshold](#)
Comparison value.
- [Priority](#) [priority](#)
Alert priority.
- `int` [consecutive_measurements](#)
Required consecutive violations (stateful only).
- `std::string` [logic](#)
"AND" or "OR"
- `std::vector< std::string >` [sub_rules](#)
Sub-rule IDs referenced by correlation.
- `int` [sensor_token_idx](#)

7.11.1 Detailed Description

In-memory representation of a single monitoring rule loaded from JSON.

7.11.2 Constructor & Destructor Documentation

7.11.2.1 RuleDefinition()

```
astralog::RuleDefinition::RuleDefinition () [inline]
```

7.11.3 Member Data Documentation

7.11.3.1 consecutive_measurements

```
int astralog::RuleDefinition::consecutive_measurements
```

Required consecutive violations (stateful only).

7.11.3.2 logic

```
std::string astralog::RuleDefinition::logic
```

"AND" or "OR"

7.11.3.3 op

```
CompOp astralog::RuleDefinition::op
```

Comparison operator.

7.11.3.4 priority

```
Priority astralog::RuleDefinition::priority
```

Alert priority.

7.11.3.5 rule_id

```
std::string astralog::RuleDefinition::rule_id
```

Unique identifier (e.g. "R001").

7.11.3.6 sensor_id

```
std::string astralog::RuleDefinition::sensor_id
```

Target sensor (empty for correlation).

7.11.3.7 sensor_token_idx

```
int astralog::RuleDefinition::sensor_token_idx
```

7.11.3.8 sub_rules

```
std::vector<std::string> astralog::RuleDefinition::sub_rules
```

Sub-rule IDs referenced by correlation.

7.11.3.9 threshold

```
double astralog::RuleDefinition::threshold
```

Comparison value.

7.11.3.10 type

```
RuleType astralog::RuleDefinition::type
```

THRESHOLD, STEP_DIFF, STATEFUL, or CORRELATION.

The documentation for this struct was generated from the following file:

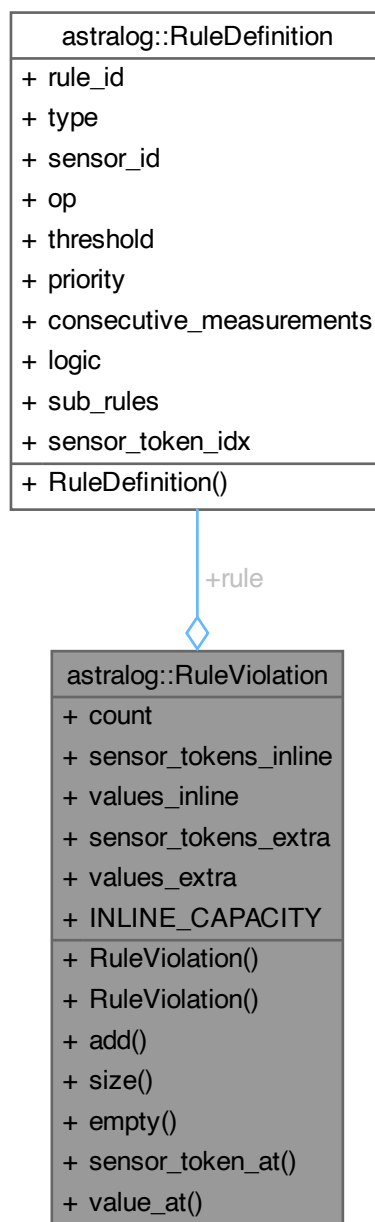
- [src/types.hpp](#)

7.12 astralog::RuleViolation Struct Reference

Represents a single rule violation at a given timestamp.

```
#include <types.hpp>
```

Collaboration diagram for astralog::RuleViolation:



Public Member Functions

- [RuleViolation](#) ()=default

- [RuleViolation](#) (const [RuleDefinition](#) *rule_ptr)
- void [add](#) (int sensor_token, double value)
- size_t [size](#) () const noexcept
- bool [empty](#) () const noexcept
- int [sensor_token_at](#) (size_t idx) const
- double [value_at](#) (size_t idx) const

Public Attributes

- const [RuleDefinition](#) * [rule](#) = nullptr
Stable pointer to loaded rule metadata.
- size_t [count](#) = 0
- std::array< int, [INLINE_CAPACITY](#) > [sensor_tokens_inline](#) {}
- std::array< double, [INLINE_CAPACITY](#) > [values_inline](#) {}
- std::vector< int > [sensor_tokens_extra](#)
- std::vector< double > [values_extra](#)

Static Public Attributes

- static constexpr size_t [INLINE_CAPACITY](#) = 4

7.12.1 Detailed Description

Represents a single rule violation at a given timestamp.

For simple/step_diff/stateful rules: one sensor_id and one value. For correlation rules: multiple sensor_ids and values from sub-rules.

The common case stores sensor/value pairs inline, avoiding per-violation heap allocation. Larger correlation rules transparently spill to vectors.

7.12.2 Constructor & Destructor Documentation

7.12.2.1 RuleViolation() [1/2]

```
astralog::RuleViolation::RuleViolation () [default]
```

7.12.2.2 RuleViolation() [2/2]

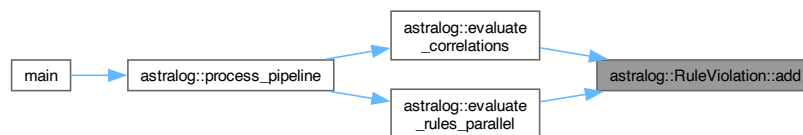
```
astralog::RuleViolation::RuleViolation (
    const RuleDefinition * rule_ptr) [inline], [explicit]
```

7.12.3 Member Function Documentation

7.12.3.1 add()

```
void astralog::RuleViolation::add (
    int sensor_token,
    double value) [inline]
```

Here is the caller graph for this function:



7.12.3.2 empty()

```
bool astralog::RuleViolation::empty () const [inline], [noexcept]
```

7.12.3.3 sensor_token_at()

```
int astralog::RuleViolation::sensor_token_at (
    size_t idx) const [inline]
```

7.12.3.4 size()

```
size_t astralog::RuleViolation::size () const [inline], [noexcept]
```

7.12.3.5 value_at()

```
double astralog::RuleViolation::value_at (
    size_t idx) const [inline]
```

7.12.4 Member Data Documentation

7.12.4.1 count

```
size_t astralog::RuleViolation::count = 0
```

7.12.4.2 INLINE_CAPACITY

```
size_t astralog::RuleViolation::INLINE_CAPACITY = 4 [static], [constexpr]
```

7.12.4.3 rule

```
const RuleDefinition* astralog::RuleViolation::rule = nullptr
```

Stable pointer to loaded rule metadata.

7.12.4.4 sensor_tokens_extra

```
std::vector<int> astralog::RuleViolation::sensor_tokens_extra
```

7.12.4.5 sensor_tokens_inline

```
std::array<int, INLINE_CAPACITY> astralog::RuleViolation::sensor_tokens_inline {}
```

7.12.4.6 values_extra

```
std::vector<double> astralog::RuleViolation::values_extra
```

7.12.4.7 values_inline

```
std::array<double, INLINE_CAPACITY> astralog::RuleViolation::values_inline {}
```

The documentation for this struct was generated from the following file:

- [src/types.hpp](#)

7.13 astralog::SensorRuleSet Struct Reference

Pre-built lookup table mapping a sensor token to its applicable rules, sorted by priority within each type.

```
#include <timestamp_processor.hpp>
```

Collaboration diagram for astralog::SensorRuleSet:

astralog::SensorRuleSet	
+	threshold
+	step_diff
+	stateful

Classes

- struct [StatefulEntry](#)

Public Attributes

- std::vector< const [RuleDefinition](#) * > [threshold](#)
- std::vector< const [RuleDefinition](#) * > [step_diff](#)
- std::vector< [StatefulEntry](#) > [stateful](#)

7.13.1 Detailed Description

Pre-built lookup table mapping a sensor token to its applicable rules, sorted by priority within each type.

7.13.2 Member Data Documentation**7.13.2.1 stateful**

```
std::vector<StatefulEntry> astralog::SensorRuleSet::stateful
```

7.13.2.2 step_diff

```
std::vector<const RuleDefinition*> astralog::SensorRuleSet::step_diff
```

7.13.2.3 threshold

```
std::vector<const RuleDefinition*> astralog::SensorRuleSet::threshold
```

The documentation for this struct was generated from the following file:

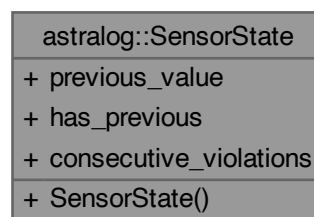
- src/[timestamp_processor.hpp](#)

7.14 astralog::SensorState Struct Reference

Per-sensor mutable state for sequential rule evaluation.

```
#include <types.hpp>
```

Collaboration diagram for astralog::SensorState:



Public Member Functions

- [SensorState](#) ()

Public Attributes

- double [previous_value](#)
Last observed reading.
- bool [has_previous](#)
True after first reading.
- std::vector< int > [consecutive_violations](#)
Per-stateful-rule counters.

7.14.1 Detailed Description

Per-sensor mutable state for sequential rule evaluation.

Cache-line aligned (64 bytes) to avoid false sharing in OpenMP parallel-for loops when different threads process adjacent sensors.

7.14.2 Constructor & Destructor Documentation

7.14.2.1 SensorState()

```
astralog::SensorState::SensorState () [inline]
```

7.14.3 Member Data Documentation

7.14.3.1 consecutive_violations

```
std::vector<int> astralog::SensorState::consecutive_violations
```

Per-stateful-rule counters.

7.14.3.2 has_previous

```
bool astralog::SensorState::has_previous
```

True after first reading.

7.14.3.3 previous_value

```
double astralog::SensorState::previous_value
```

Last observed reading.

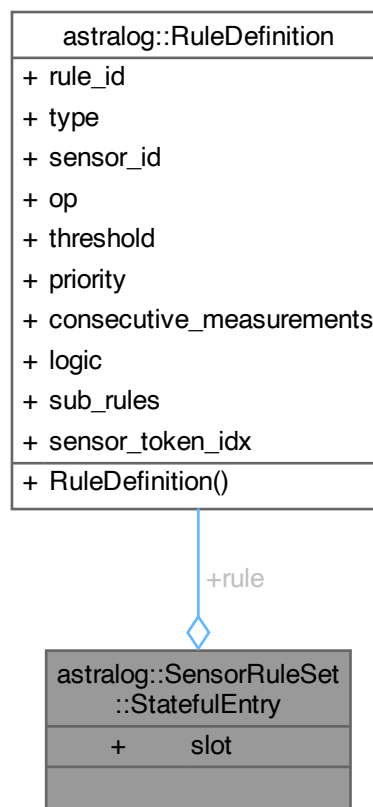
The documentation for this struct was generated from the following file:

- [src/types.hpp](#)

7.15 astralog::SensorRuleSet::StatefulEntry Struct Reference

```
#include <timestamp_processor.hpp>
```

Collaboration diagram for astralog::SensorRuleSet::StatefulEntry:



Public Attributes

- const [RuleDefinition](#) * `rule`
- int `slot`

Index into [SensorState::consecutive_violations\[\]](#).

7.15.1 Member Data Documentation

7.15.1.1 rule

```
const RuleDefinition* astralog::SensorRuleSet::StatefulEntry::rule
```

7.15.1.2 slot

```
int astralog::SensorRuleSet::StatefulEntry::slot
```

Index into [SensorState::consecutive_violations\[\]](#).

The documentation for this struct was generated from the following file:

- [src/timestamp_processor.hpp](#)

7.16 astralog::TimestampGroup Struct Reference

Groups all valid sensor readings that share the same timestamp.

```
#include <types.hpp>
```

Collaboration diagram for `astralog::TimestampGroup`:

astralog::TimestampGroup	
+	timestamp
+	readings

Classes

- struct [Reading](#)

Public Attributes

- `std::string` [timestamp](#)
- `std::vector< Reading >` [readings](#)

7.16.1 Detailed Description

Groups all valid sensor readings that share the same timestamp.

Used for per-timestamp NOMINAL/ANOMALOUS decision logic.

7.16.2 Member Data Documentation

7.16.2.1 readings

```
std::vector<Reading> astralog::TimestampGroup::readings
```

7.16.2.2 timestamp

```
std::string astralog::TimestampGroup::timestamp
```

The documentation for this struct was generated from the following file:

- [src/types.hpp](#)

7.17 astralog::TokenViewEntry Struct Reference

```
#include <csv_parser.hpp>
```

Collaboration diagram for `astralog::TokenViewEntry`:

astralog::TokenViewEntry	
+	sensor_id
+	token

Public Attributes

- `std::string_view` [sensor_id](#)
- `int` [token](#)

7.17.1 Member Data Documentation

7.17.1.1 sensor_id

```
std::string_view astralog::TokenViewEntry::sensor_id
```

7.17.1.2 token

```
int astralog::TokenViewEntry::token
```

The documentation for this struct was generated from the following file:

- [src/csv_parser.hpp](#)

Chapter 8

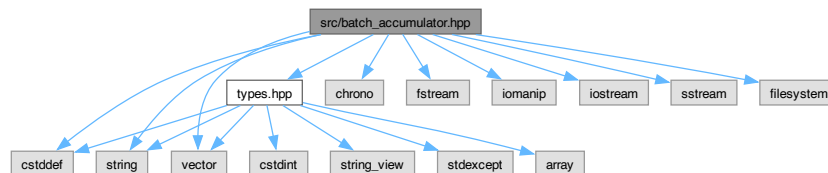
File Documentation

8.1 src/batch_accumulator.hpp File Reference

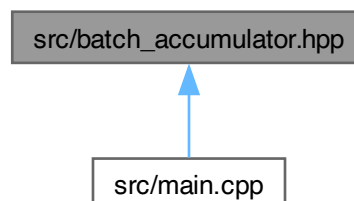
Batch Accumulator component for AstraLog-HPC.

```
#include <cstdint>
#include <chrono>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <sstream>
#include <string>
#include <vector>
#include <filesystem>
#include "types.hpp"
```

Include dependency graph for batch_accumulator.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- struct [astralog::BatchConfig](#)
Configuration for the batch accumulator.
- class [astralog::BatchAccumulator](#)
Accumulates valid records and flushes them in configurable batches.

Namespaces

- namespace [astralog](#)

Enumerations

- enum class [astralog::BatchStrategy](#) : uint8_t { [astralog::COUNT](#) = 0 , [astralog::TIME](#) = 1 }
Batch flushing strategy.

Functions

- const char * [astralog::strategy_to_string](#) (BatchStrategy s) noexcept
- BatchStrategy [astralog::string_to_strategy](#) (const std::string &s)

8.1.1 Detailed Description

Batch Accumulator component for AstraLog-HPC.

Sits between CSV validation and the rule evaluation pipeline. Accumulates valid ParsedRecord objects and flushes them in batches using one of two strategies:

- COUNT: Flush every N valid records
- TIME: Flush every N milliseconds of wall-clock time

Each flush writes a timestamped audit file to disk and returns the batch of records for processing by the rule engine. This component matches the Batch Accumulator described in the Phase 1 design document (Section 3.1.1, FR.3, FR.7, Use Case 3).

8.1.1.1 Design Notes

- The accumulator operates in memory; the .txt audit file is written as a side-effect of flush() for traceability and debugging, unless disabled for benchmark runs.
- Time-based strategy uses wall-clock elapsed time from the moment the current batch started accumulating. On small files the entire file may fit in a single batch.
- Thread safety is NOT required: the accumulator is called from the sequential main loop (between parallel CSV parse and parallel rule evaluation).

8.2 batch_accumulator.hpp

[Go to the documentation of this file.](#)

```

00001
00029
00030 #ifndef ASTRALOG_BATCH_ACCUMULATOR_HPP
00031 #define ASTRALOG_BATCH_ACCUMULATOR_HPP
00032
00033 #include <cstdint>
00034 #include <chrono>
00035 #include <fstream>
00036 #include <iomanip>
00037 #include <iostream>
00038 #include <sstream>
00039 #include <string>
00040 #include <vector>
00041 #include <filesystem>
00042
00043 #include "types.hpp"
00044
00045 namespace astralog {
00046
00047 // =====
00048 // Batch strategy enumeration
00049 // =====
00050
00052 enum class BatchStrategy : uint8_t {
00053     COUNT = 0,
00054     TIME = 1
00055 };
00056
00057 inline const char* strategy_to_string(BatchStrategy s) noexcept {
00058     switch (s) {
00059         case BatchStrategy::COUNT: return "COUNT";
00060         case BatchStrategy::TIME: return "TIME";
00061         default: return "UNKNOWN";
00062     }
00063 }
00064
00065 inline BatchStrategy string_to_strategy(const std::string& s) {
00066     if (s == "count" || s == "COUNT") return BatchStrategy::COUNT;
00067     if (s == "time" || s == "TIME") return BatchStrategy::TIME;
00068     throw std::invalid_argument("Unknown batch strategy: " + s);
00069 }
00070
00071 // =====
00072 // Batch configuration
00073 // =====
00074
00079 struct BatchConfig {
00080     BatchStrategy strategy;
00081     size_t count_threshold;
00082     double time_threshold_ms;
00083     std::string batch_output_dir;
00084     bool write_audit_files;
00085
00086     BatchConfig()
00087         : strategy(BatchStrategy::COUNT)
00088         , count_threshold(1000)
00089         , time_threshold_ms(5000.0)
00090         , batch_output_dir("output/batches")
00091         , write_audit_files(true)
00092     {}
00093 };
00094
00095 // =====
00096 // Batch Accumulator
00097 // =====
00098
00117 class BatchAccumulator {
00118 public:
00119     explicit BatchAccumulator(const BatchConfig& config,
00120                               const std::vector<std::string>& inverse_token_map)
00121         : config_(config)
00122         , inverse_token_map_(inverse_token_map)
00123         , batch_number_(0)
00124         , batch_start_time_(std::chrono::steady_clock::now())
00125     {
00126         buffer_.reserve(config.count_threshold > 0 ? config.count_threshold : 1000);
00127     }
00128
00132 void add_record(ParsedRecord rec) {
00133     buffer_.push_back(std::move(rec));
00134 }
00135

```

```

00142     bool should_flush() const {
00143         if (buffer_.empty()) return false;
00144
00145         switch (config_.strategy) {
00146             case BatchStrategy::COUNT:
00147                 return buffer_.size() >= config_.count_threshold;
00148
00149             case BatchStrategy::TIME: {
00150                 auto now = std::chrono::steady_clock::now();
00151                 double elapsed_ms = std::chrono::duration<double, std::milli>(
00152                     now - batch_start_time_).count();
00153                 return elapsed_ms >= config_.time_threshold_ms;
00154             }
00155
00156             default:
00157                 return false;
00158         }
00159     }
00160
00161     std::vector<ParsedRecord> flush() {
00162         ++batch_number_;
00163
00164         std::vector<ParsedRecord> batch;
00165         batch.swap(buffer_);
00166
00167         if (config_.write_audit_files) {
00168             write_batch_file(batch);
00169         }
00170
00171         // Reset timer for next batch (TIME strategy)
00172         batch_start_time_ = std::chrono::steady_clock::now();
00173
00174         // Re-reserve buffer for next batch
00175         buffer_.reserve(config_.count_threshold > 0 ? config_.count_threshold : 1000);
00176
00177         return batch;
00178     }
00179
00180     std::vector<ParsedRecord> flush_remaining() {
00181         if (buffer_.empty()) return {};
00182         return flush();
00183     }
00184
00185     int batch_count() const noexcept {
00186         return batch_number_;
00187     }
00188
00189     size_t buffered_count() const noexcept {
00190         return buffer_.size();
00191     }
00192
00193     const BatchConfig& config() const noexcept {
00194         return config_;
00195     }
00196
00197 private:
00198     BatchConfig config_;
00199     std::vector<std::string> inverse_token_map_;
00200     std::vector<ParsedRecord> buffer_;
00201     int batch_number_;
00202     std::chrono::steady_clock::time_point batch_start_time_;
00203
00204     void write_batch_file(const std::vector<ParsedRecord>& batch) {
00205         namespace fs = std::filesystem;
00206
00207         try {
00208             fs::create_directories(config_.batch_output_dir);
00209         } catch (const std::exception& e) {
00210             std::cerr << "[batch] WARNING: cannot create batch output dir: "
00211                 << e.what() << "\n";
00212             return;
00213         }
00214
00215         // Build filename: batch_001_20251115_120000.txt
00216         auto now = std::chrono::system_clock::now();
00217         auto now_t = std::chrono::system_clock::to_time_t(now);
00218         std::tm tm_buf{};
00219         #if defined(_WIN32)
00220             gmtime_s(&tm_buf, &now_t);
00221         #else
00222             gmtime_r(&now_t, &tm_buf);
00223         #endif
00224
00225         std::ostringstream fname;
00226         fname << "batch_"
00227             << std::setfill('0') << std::setw(3) << batch_number_
00228             << "_"

```

```

00260         « std::setfill('0') « std::setw(4) « (tm_buf.tm_year + 1900)
00261         « std::setfill('0') « std::setw(2) « (tm_buf.tm_mon + 1)
00262         « std::setfill('0') « std::setw(2) « tm_buf.tm_mday
00263         « "-"
00264         « std::setfill('0') « std::setw(2) « tm_buf.tm_hour
00265         « std::setfill('0') « std::setw(2) « tm_buf.tm_min
00266         « std::setfill('0') « std::setw(2) « tm_buf.tm_sec
00267         « ".txt";
00268
00269     std::string filepath = (fs::path(config_.batch_output_dir) / fname.str()).string();
00270
00271     std::ofstream ofs(filepath, std::ios::out | std::ios::trunc);
00272     if (!ofs.is_open()) {
00273         std::cerr « "[batch] WARNING: cannot open batch file: "
00274                 « filepath « "\n";
00275         return;
00276     }
00277
00278     // Header comments
00279     ofs « "# Batch " « std::setfill('0') « std::setw(3) « batch_number_
00280         « " | Strategy: " « strategy_to_string(config_.strategy)
00281         « " | Size: " « batch.size() « " records\n";
00282
00283     ofs « "# Flushed at: "
00284         « std::setfill('0') « std::setw(4) « (tm_buf.tm_year + 1900) « "-"
00285         « std::setfill('0') « std::setw(2) « (tm_buf.tm_mon + 1) « "-"
00286         « std::setfill('0') « std::setw(2) « tm_buf.tm_mday « "T"
00287         « std::setfill('0') « std::setw(2) « tm_buf.tm_hour « ":"
00288         « std::setfill('0') « std::setw(2) « tm_buf.tm_min « ":"
00289         « std::setfill('0') « std::setw(2) « tm_buf.tm_sec « "Z\n";
00290
00291     // CSV header
00292     ofs « "timestamp,sensor_id,value\n";
00293
00294     // Data rows
00295     for (const auto& rec : batch) {
00296         ofs « rec.timestamp « ",";
00297         if (rec.sensor_token >= 0 &&
00298             static_cast<size_t>(rec.sensor_token) < inverse_token_map_.size()) {
00299             ofs « inverse_token_map_[rec.sensor_token];
00300         } else {
00301             ofs « "UNKNOWN";
00302         }
00303         ofs « "," « rec.value « "\n";
00304     }
00305
00306     ofs.close();
00307 }
00308 };
00309
00310 } // namespace astralog
00311
00312 #endif // ASTRALOG_BATCH_ACCUMULATOR_HPP

```

8.3 src/csv_parser.hpp File Reference

Memory-mapped CSV parser with parallel line parsing.

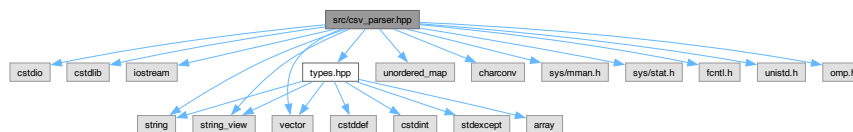
```

#include <cstdio>
#include <cstdlib>
#include <iostream>
#include <string>
#include <string_view>
#include <vector>
#include <unordered_map>
#include <charconv>
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <omp.h>

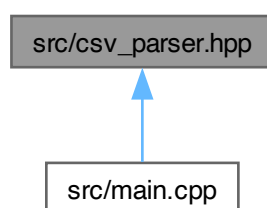
```

```
#include "types.hpp"
```

Include dependency graph for csv_parser.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- struct [astralog::TokenViewEntry](#)
- struct [astralog::MappedFile](#)

RAII wrapper for a read-only memory-mapped file.

Namespaces

- namespace [astralog](#)

Macros

- `#define` [MAP_POPULATE](#) 0

Typedefs

- using [astralog::TokenViewMap](#) = `std::vector<TokenViewEntry>`

Functions

- `std::vector< std::string > astralog::build_inverse_token_map` (const `std::unordered_map< std::string, int > &token_map`)
Build an inverse token map: `sensor_token (int) → sensor_id (string)`.
- `TokenViewMap astralog::build_token_view_map` (const `std::vector< std::string > &inverse_token_map`)
Build a `string_view` token lookup table backed by `inverse_token_map` storage.
- `int astralog::lookup_sensor_token` (const `TokenViewMap &token_map`, `std::string_view sensor_id`) noexcept
- `std::vector< size_t > astralog::build_line_offsets` (const `char *data`, `size_t size`)
Build a vector of byte offsets for each data line (skipping header).
- `std::string_view astralog::get_line` (const `char *data`, `size_t size`, `size_t offset`) noexcept
Extract a line as `string_view` from mmap data at the given offset.
- `bool astralog::tokenize_csv_line` (`std::string_view line`, `std::string_view &f0`, `std::string_view &f1`, `std::string_view &f2`, `std::string_view &f3`) noexcept
Tokenize a CSV line into exactly 4 comma-separated fields.
- `bool astralog::validate_schema` (`std::string_view f0`, `std::string_view f1`, `std::string_view f2`, `std::string_view f3`) noexcept
ESA-compliant schema validation.
- `bool astralog::parse_value` (`std::string_view field`, `double &val`) noexcept
Parse a value field to double using `std::from_chars` (C++17). Non-allocating, exception-free.
- `std::vector< ParsedRecord > astralog::parse_csv_parallel` (const `MappedFile &csv_file`, const `std::vector< size_t > &line_offsets`, const `TokenViewMap &token_map`, `uint64_t &total_valid`, `uint64_t &total_invalid`)
Parse the entire CSV file in parallel, returning validated records in original CSV line order.

8.3.1 Detailed Description

Memory-mapped CSV parser with parallel line parsing.

Pipeline:

1. Memory-map the CSV file using POSIX `mmap(2)`
2. Sequential pre-scan to build line offset index ($O(1)$ random access)
3. OpenMP parallel-for to parse and validate lines concurrently
4. Merge thread-local results into a single ordered vector

Validation (ESA-compliant):

- Malformed lines: not exactly 4 comma-separated fields
- Schema errors: empty fields, "ERR" or "CORRUPT" markers
- Type errors: non-numeric value field

8.3.2 Macro Definition Documentation

8.3.2.1 MAP_POPULATE

```
#define MAP_POPULATE 0
```

8.4 csv_parser.hpp

[Go to the documentation of this file.](#)

```

00001
00016
00017 #ifndef ASTRALOG_CSV_PARSER_HPP
00018 #define ASTRALOG_CSV_PARSER_HPP
00019
00020 #include <cstdio>
00021 #include <cstdlib>
00022 #include <iostream>
00023 #include <string>
00024 #include <string_view>
00025 #include <vector>
00026 #include <unordered_map>
00027 #include <charconv>
00028
00029 // POSIX mmap
00030 #include <sys/mman.h>
00031 #include <sys/stat.h>
00032 #include <fcntl.h>
00033 #include <unistd.h>
00034
00035 // OpenMP
00036 #include <omp.h>
00037
00038 #include "types.hpp"
00039
00040 // Portability: MAP_POPULATE is Linux-specific (prefaults page tables)
00041 #ifndef MAP_POPULATE
00042 #define MAP_POPULATE 0
00043 #endif
00044
00045 namespace astralog {
00046
00047 struct TokenViewEntry {
00048     std::string_view sensor_id;
00049     int token;
00050 };
00051
00052 using TokenViewMap = std::vector<TokenViewEntry>;
00053
00054 // =====
00055 // Inverse token map utility
00056 // =====
00057
00064 inline std::vector<std::string> build_inverse_token_map(
00065     const std::unordered_map<std::string, int>& token_map)
00066 {
00067     std::vector<std::string> inverse(token_map.size());
00068     for (const auto& [name, idx] : token_map) {
00069         if (idx >= 0 && static_cast<size_t>(idx) < inverse.size())
00070             inverse[idx] = name;
00071     }
00072     return inverse;
00073 }
00074
00081 inline TokenViewMap build_token_view_map(
00082     const std::vector<std::string>& inverse_token_map)
00083 {
00084     TokenViewMap token_view_map;
00085     token_view_map.reserve(inverse_token_map.size());
00086
00087     for (size_t i = 0; i < inverse_token_map.size(); ++i) {
00088         if (!inverse_token_map[i].empty()) {
00089             token_view_map.push_back({
00090                 std::string_view(inverse_token_map[i].data(),
00091                     inverse_token_map[i].size()),
00092                 static_cast<int>(i)
00093             });
00094         }
00095     }
00096
00097     return token_view_map;
00098 }
00099
00100 inline int lookup_sensor_token(const TokenViewMap& token_map,
00101     std::string_view sensor_id) noexcept
00102 {
00103     for (const auto& entry : token_map) {
00104         if (entry.sensor_id == sensor_id) {
00105             return entry.token;
00106         }
00107     }
00108     return -1;

```

```

00109 }
00110
00111 // =====
00112 // Memory-mapped file
00113 // =====
00114
00115 struct MappedFile {
00116     const char* data = nullptr;
00117     size_t size = 0;
00118     int fd = -1;
00119
00120     ~MappedFile() {
00121         if (data && data != MAP_FAILED)
00122             munmap(const_cast<char*>(data), size);
00123         if (fd >= 0)
00124             close(fd);
00125     }
00126
00127     bool open(const char* path) {
00128         fd = ::open(path, O_RDONLY);
00129         if (fd < 0) {
00130             std::cerr << "[mmap] ERROR: cannot open file: " << path << "\n";
00131             return false;
00132         }
00133
00134         struct stat st;
00135         if (fstat(fd, &st) < 0) {
00136             std::cerr << "[mmap] ERROR: cannot stat file: " << path << "\n";
00137             close(fd); fd = -1;
00138             return false;
00139         }
00140
00141         size = static_cast<size_t>(st.st_size);
00142         data = static_cast<const char*>(
00143             mmap(nullptr, size, PROT_READ, MAP_PRIVATE | MAP_POPULATE, fd, 0));
00144
00145         if (data == MAP_FAILED) {
00146             std::cerr << "[mmap] ERROR: mmap failed for: " << path << "\n";
00147             data = nullptr;
00148             close(fd); fd = -1;
00149             return false;
00150         }
00151
00152         // Advise the kernel we will read sequentially
00153         madvise(const_cast<char*>(data), size, MADV_SEQUENTIAL);
00154         return true;
00155     }
00156 };
00157
00158 // =====
00159 // Line offset index
00160 // =====
00161
00162 inline std::vector<size_t> build_line_offsets(const char* data, size_t size) {
00163     std::vector<size_t> offsets;
00164     offsets.reserve(size / 50); // ~50 bytes per line estimate
00165
00166     // Skip header line
00167     size_t pos = 0;
00168     while (pos < size && data[pos] != '\n') ++pos;
00169
00170     // Record start of each data line
00171     while (pos < size) {
00172         offsets.push_back(pos);
00173         while (pos < size && data[pos] != '\n') ++pos;
00174     }
00175
00176     return offsets;
00177 }
00178
00179 inline std::string_view get_line(const char* data, size_t size,
00180                                 size_t offset) noexcept {
00181     size_t end = offset;
00182     while (end < size && data[end] != '\n') ++end;
00183     return std::string_view(data + offset, end - offset);
00184 }
00185
00186 // =====
00187 // Zero-copy CSV tokenization and validation
00188 // =====
00189
00190 inline bool tokenize_csv_line(std::string_view line,
00191                               std::string_view& f0, std::string_view& f1,
00192                               std::string_view& f2, std::string_view& f3) noexcept {
00193     // Strip trailing \r or \n

```

```

00211     while (!line.empty() && (line.back() == '\r' || line.back() == '\n'))
00212         line.remove_suffix(1);
00213     if (line.empty()) return false;
00214
00215     size_t c1 = line.find(',');
00216     if (c1 == std::string_view::npos) return false;
00217     size_t c2 = line.find(',', c1 + 1);
00218     if (c2 == std::string_view::npos) return false;
00219     size_t c3 = line.find(',', c2 + 1);
00220     if (c3 == std::string_view::npos) return false;
00221     // Reject if there is a 5th field
00222     if (line.find(',', c3 + 1) != std::string_view::npos) return false;
00223
00224     f0 = line.substr(0, c1);
00225     f1 = line.substr(c1 + 1, c2 - c1 - 1);
00226     f2 = line.substr(c2 + 1, c3 - c2 - 1);
00227     f3 = line.substr(c3 + 1);
00228     return true;
00229 }
00230
00231 inline bool validate_schema(std::string_view f0, std::string_view f1,
00232                             std::string_view f2, std::string_view f3) noexcept {
00233     if (f0.empty() || f1.empty() || f2.empty() || f3.empty()) return false;
00234     auto is_corrupt = [](std::string_view s) {
00235         return s == "ERR" || s == "CORRUPT";
00236     };
00237     return !is_corrupt(f0) && !is_corrupt(f1) && !is_corrupt(f2) && !is_corrupt(f3);
00238 }
00239
00240 inline bool parse_value(std::string_view field, double& val) noexcept {
00241     auto result = std::from_chars(field.data(), field.data() + field.size(), val);
00242     return (result.ec == std::errc{} && result.ptr == field.data() + field.size());
00243 }
00244
00245 // =====
00246 // Parallel CSV parsing
00247 // =====
00248
00249 inline std::vector<ParsedRecord> parse_csv_parallel(
00250     const MappedFile& csv_file,
00251     const std::vector<size_t>& line_offsets,
00252     const TokenViewMap& token_map,
00253     uint64_t& total_valid,
00254     uint64_t& total_invalid)
00255 {
00256     const size_t total_lines = line_offsets.size();
00257     const int num_threads = omp_get_max_threads();
00258
00259     std::vector<std::vector<ParsedRecord>> thread_records(num_threads);
00260     uint64_t valid_count = 0, invalid_count = 0;
00261
00262     #pragma omp parallel reduction(+:valid_count, invalid_count)
00263     {
00264         int tid = omp_get_thread_num();
00265         thread_records[tid].reserve(total_lines / num_threads + 1);
00266
00267         #pragma omp for schedule(static)
00268         for (size_t i = 0; i < total_lines; ++i) {
00269             std::string_view line = get_line(csv_file.data, csv_file.size,
00270                                             line_offsets[i]);
00271
00272             std::string_view f_ts, f_sensor, f_value, f_priority;
00273             if (!tokenize_csv_line(line, f_ts, f_sensor, f_value, f_priority)) {
00274                 ++invalid_count;
00275                 continue;
00276             }
00277             if (!validate_schema(f_ts, f_sensor, f_value, f_priority)) {
00278                 ++invalid_count;
00279                 continue;
00280             }
00281             double value;
00282             if (!parse_value(f_value, value)) {
00283                 ++invalid_count;
00284                 continue;
00285             }
00286
00287             ++valid_count;
00288
00289             ParsedRecord rec;
00290             rec.timestamp = std::string(f_ts);
00291             rec.value = value;
00292
00293             // Tiny fixed lookup table: avoids allocating and avoids hash cost.
00294             rec.sensor_token = lookup_sensor_token(token_map, f_sensor);
00295
00296             thread_records[tid].push_back(std::move(rec));
00297         }
00298     }
00299 }

```

```

00322     }
00323
00324     total_valid    = valid_count;
00325     total_invalid = invalid_count;
00326
00327     // Merge thread-local vectors in thread order → preserves CSV line order
00328     std::vector<ParsedRecord> all_records;
00329     size_t total_size = 0;
00330     for (int t = 0; t < num_threads; ++t) total_size += thread_records[t].size();
00331     all_records.reserve(total_size);
00332
00333     for (int t = 0; t < num_threads; ++t) {
00334         for (auto& rec : thread_records[t]) {
00335             all_records.push_back(std::move(rec));
00336         }
00337     }
00338
00339     return all_records;
00340 }
00341
00342 } // namespace astralog
00343
00344 #endif // ASTRALOG_CSV_PARSER_HPP

```

8.5 src/main.cpp File Reference

AstraLog-HPC — Entry point and orchestration.

```

#include <iostream>
#include <fstream>
#include <iomanip>
#include <string>
#include <vector>
#include <unordered_map>
#include <filesystem>
#include <chrono>
#include <omp.h>
#include "types.hpp"
#include "csv_parser.hpp"
#include "yaml_parser.hpp"
#include "rules_engine.hpp"
#include "output_formatter.hpp"
#include "batch_accumulator.hpp"
#include "timestamp_processor.hpp"

```

Include dependency graph for main.cpp:



Functions

- int [main](#) (int argc, char *argv[])

8.5.1 Detailed Description

AstraLog-HPC — Entry point and orchestration.

This file is a thin orchestrator that:

1. Parses command-line arguments (including batch configuration)
2. Loads configuration (sensors YAML, rules JSON)
3. Memory-maps the CSV telemetry file
4. Feeds valid records through the BatchAccumulator
5. Delegates each batch to the modular processing pipeline
6. Writes output files (valid_data.csv, alarms.log, batch audit files)

All heavy lifting is in the header modules:

- [csv_parser.hpp](#): Parallel CSV parsing with mmap
- [yaml_parser.hpp](#): Sensor configuration loading
- [rules_engine.hpp](#): Rule loading and evaluation functions
- [batch_accumulator.hpp](#): Count/Time batch accumulation strategy
- [timestamp_processor.hpp](#): 5-phase parallel processing pipeline
- [output_formatter.hpp](#): Spec-compliant output formatting

Build: `mkdir build && cd build && cmake .. -DCMAKE_BUILD_TYPE=Release && make`

Run: `./build/bin/astralog_processing -csv input/telemetry/export_sat_alpha_small.csv`

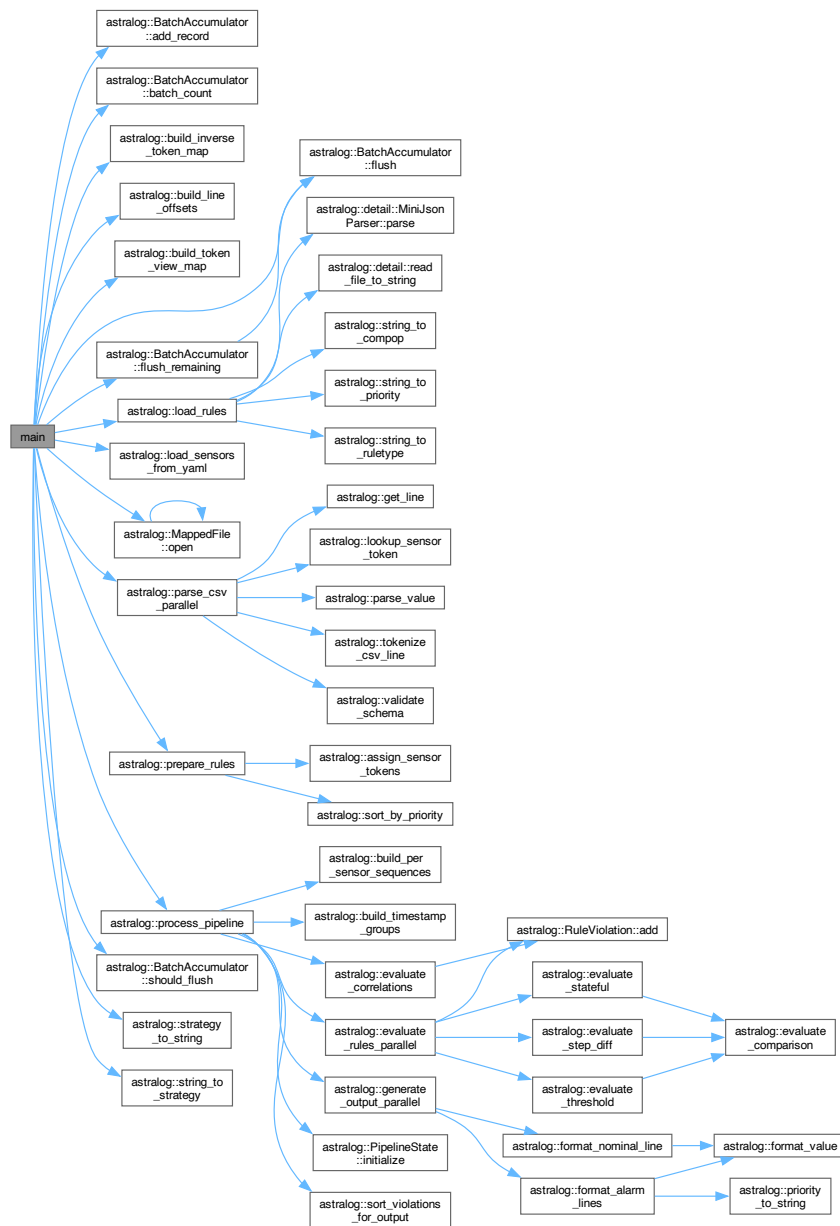
8.5.2 Function Documentation

8.5.2.1 main()

```
int main (  
    int argc,  
    char * argv[])
```

Process a single batch through the rule evaluation pipeline and append results to the output files.

Lambda captures output file streams and pipeline state by reference so that sensor state persists across batches. Here is the call graph for this function:



8.6 src/output_formatter.hpp File Reference

Spec-compliant output formatting for valid_data.csv and alarms.log.

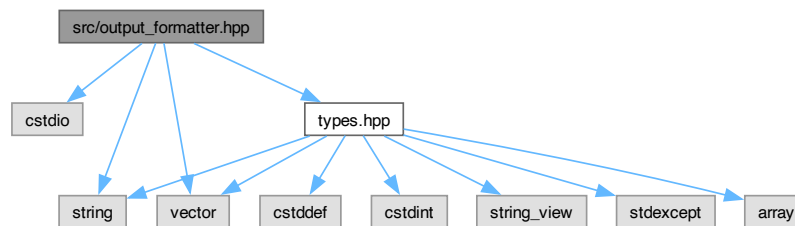
```

#include <stdio>
#include <string>
#include <vector>

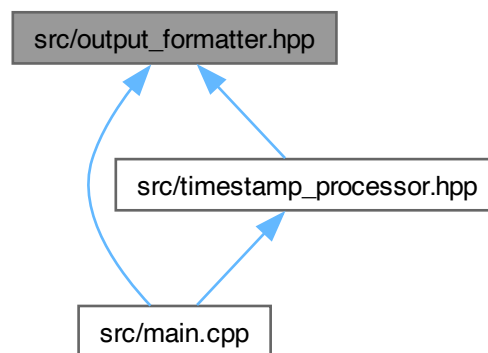
```

```
#include "types.hpp"
```

Include dependency graph for output_formatter.hpp:



This graph shows which files directly or indirectly include this file:



Namespaces

- namespace [astralog](#)

Functions

- `std::string astralog::format\_value` (double val)
Format a double to string, stripping trailing zeros after the decimal point but keeping at least one decimal digit.
- `std::string astralog::format\_nominal\_line` (const [TimestampGroup](#) &group, const `std::vector< std::string >` &inverse_token_map)
Format a NOMINAL line for valid_data.csv.
- `std::string astralog::format\_alarm\_lines` (const `std::string` ×tamp, const `std::vector< RuleViolation >` &violations, const `std::vector< std::string >` &inverse_token_map)
Format alarm lines for alarms.log.

8.6.1 Detailed Description

Spec-compliant output formatting for valid_data.csv and alarms.log.

Output formats (bare semicolon separators, per ESA spec):

valid_data.csv (NOMINAL timestamps): `TIMESTAMP;NOMINAL;SENSOR_1:VALUE_1|SENSOR_2:VALUE_2|...`

alarms.log (ANOMALOUS timestamps): `TIMESTAMP;RULE_ID;PRIORITY;SENSOR(S);VALUE(S)`

For correlation rules, sensors and values are comma separated: `TIMESTAMP;R4;HIGH;TEMP-01,PRES-01;51.2,98.0`

8.7 output_formatter.hpp

[Go to the documentation of this file.](#)

```
00001
00016
00017 #ifndef ASTRALOG_OUTPUT_FORMATTER_HPP
00018 #define ASTRALOG_OUTPUT_FORMATTER_HPP
00019
00020 #include <cstdio>
00021 #include <string>
00022 #include <vector>
00023
00024 #include "types.hpp"
00025
00026 namespace astralog {
00027
00028 // =====
00029 // Value formatting
00030 // =====
00031
00032 inline std::string format_value(double val) {
00033     char buf[64];
00034     std::snprintf(buf, sizeof(buf), "%.6f", val);
00035     std::string s(buf);
00036     size_t dot = s.find('.');
00037     if (dot != std::string::npos) {
00038         size_t last_nonzero = s.find_last_not_of('0');
00039         if (last_nonzero != std::string::npos && last_nonzero > dot) {
00040             s.erase(last_nonzero + 1);
00041         } else if (last_nonzero == dot) {
00042             s.erase(dot + 2); // Keep at least "X.0"
00043         }
00044     }
00045     return s;
00046 }
00047
00048 // =====
00049 // NOMINAL line formatting
00050 // =====
00051
00052 inline std::string format_nominal_line(
00053     const TimestampGroup& group,
00054     const std::vector<std::string>& inverse_token_map)
00055 {
00056     std::string line;
00057     line.reserve(128);
00058
00059     line += group.timestamp;
00060     line += ";NOMINAL;";
00061
00062     for (size_t i = 0; i < group.readings.size(); ++i) {
00063         if (i > 0) line += '|';
00064         int token = group.readings[i].sensor_token;
00065         if (token >= 0 && static_cast<size_t>(token) < inverse_token_map.size()) {
00066             line += inverse_token_map[token];
00067         } else {
00068             line += "UNKNOWN";
00069         }
00070         line += ':';
00071         line += format_value(group.readings[i].value);
00072     }
00073 }
```

```

00089     line += '\n';
00090     return line;
00091 }
00092
00093 // =====
00094 // Alarm line formatting
00095 // =====
00096
00111 inline std::string format_alarm_lines(
00112     const std::string& timestamp,
00113     const std::vector<RuleViolation>& violations,
00114     const std::vector<std::string>& inverse_token_map)
00115 {
00116     std::string lines;
00117     lines.reserve(violations.size() * 80);
00118
00119     for (const auto& v : violations) {
00120         lines += timestamp;
00121         lines += ',';
00122         lines += v.rule ? v.rule->rule_id : "UNKNOWN_RULE";
00123         lines += ',';
00124         lines += priority_to_string(v.rule ? v.rule->priority
00125                                     : Priority::LOW);
00126         lines += ',';
00127
00128         // Sensor names (comma separated for correlation)
00129         for (size_t i = 0; i < v.size(); ++i) {
00130             if (i > 0) lines += ',';
00131             int token = v.sensor_token_at(i);
00132             if (token >= 0 && static_cast<size_t>(token) < inverse_token_map.size()) {
00133                 lines += inverse_token_map[token];
00134             } else {
00135                 lines += "UNKNOWN";
00136             }
00137         }
00138
00139         lines += ',';
00140
00141         // Values (comma separated for correlation)
00142         for (size_t i = 0; i < v.size(); ++i) {
00143             if (i > 0) lines += ',';
00144             lines += format_value(v.value_at(i));
00145         }
00146
00147         lines += '\n';
00148     }
00149
00150     return lines;
00151 }
00152
00153 } // namespace astralog
00154
00155 #endif // ASTRALOG_OUTPUT_FORMATTER_HPP

```

8.8 src/rules_engine.hpp File Reference

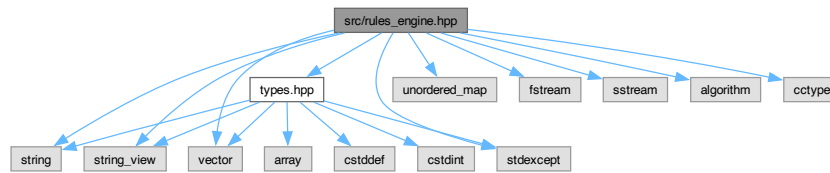
Rule loading from JSON and inline rule evaluation functions.

```

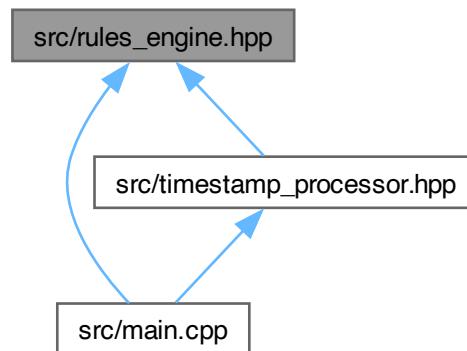
#include <string>
#include <string_view>
#include <vector>
#include <unordered_map>
#include <fstream>
#include <sstream>
#include <stdexcept>
#include <algorithm>
#include <cctype>
#include "types.hpp"

```

Include dependency graph for rules_engine.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [astralog::detail::MiniJsonParser](#)
- struct [astralog::detail::MiniJsonParser::JsonValue](#)

Namespaces

- namespace [astralog](#)
- namespace [astralog::detail](#)

Functions

- `std::string` [astralog::detail::read_file_to_string](#) (const std::string &filepath)
- `std::vector< RuleDefinition >` [astralog::load_rules](#) (const std::string &filepath)
Load monitoring rules from a JSON file.
- void [astralog::sort_by_priority](#) (std::vector< const RuleDefinition * > &rules)
Sort a vector of rule pointers by priority (HIGH first, then MEDIUM, then LOW).
- void [astralog::assign_sensor_tokens](#) (std::vector< RuleDefinition > &rules, const std::unordered_map< std::string, int > &token_map)

- Assign sensor_token_idx to each non-correlation rule based on the provided token map.*
- bool `astralog::evaluate_threshold` (double value, const `RuleDefinition` &rule) noexcept
Evaluate a THRESHOLD (simple) rule: value <op> threshold.
 - bool `astralog::evaluate_step_diff` (double value, const `SensorState` &state, const `RuleDefinition` &rule) noexcept
Evaluate a STEP_DIFF rule using SIGNED delta.
 - bool `astralog::evaluate_stateful` (double value, `SensorState` &state, const `RuleDefinition` &rule, int rule_slot) noexcept
Evaluate a STATEFUL rule: consecutive violation persistence.

8.8.1 Detailed Description

Rule loading from JSON and inline rule evaluation functions.

Components:

- `MiniJsonParser`: dependency-free JSON parser for rules.json
- `load_rules()`: deserializes JSON array → `vector<RuleDefinition>`
- `sort_rules_by_priority()`: ensures HIGH→MEDIUM→LOW evaluation order
- `evaluate_threshold()`: simple absolute threshold check
- `evaluate_step_diff()`: signed delta between consecutive readings
- `evaluate_stateful()`: consecutive violation persistence counter

Design notes:

- `step_diff` uses SIGNED delta (current - previous), NOT `abs()`. This matches the spec: operator "<" with value -2.0 means "delta < -2.0", detecting drops greater than 2 units.
- All evaluators are inline and non-allocating on the hot path.

8.9 rules_engine.hpp

[Go to the documentation of this file.](#)

```
00001
00019
00020 #ifndef ASTRALOG_RULES_ENGINE_HPP
00021 #define ASTRALOG_RULES_ENGINE_HPP
00022
00023 #include <string>
00024 #include <string_view>
00025 #include <vector>
00026 #include <unordered_map>
00027 #include <fstream>
00028 #include <sstream>
00029 #include <stdexcept>
00030 #include <algorithm>
00031 #include <cctype>
00032
00033 #include "types.hpp"
00034
00035 namespace astralog {
00036
00037 // =====
00038 // Minimal JSON parser (dependency-free)
00039 // =====
00040
```

```

00041 namespace detail {
00042
00043 class MiniJsonParser {
00044 public:
00045     struct JsonValue {
00046         enum Type { STRING, NUMBER, ARRAY, OBJECT, NONE } type = NONE;
00047         std::string str_val;
00048         double num_val = 0.0;
00049         std::vector<JsonValue> arr_val;
00050         std::vector<std::pair<std::string, JsonValue>> obj_val;
00051
00052         const JsonValue* find(const std::string& key) const {
00053             for (const auto& kv : obj_val)
00054                 if (kv.first == key) return &kv.second;
00055             return nullptr;
00056         }
00057         std::string as_string(const std::string& def = "") const {
00058             return (type == STRING) ? str_val : def;
00059         }
00060         double as_number(double def = 0.0) const {
00061             return (type == NUMBER) ? num_val : def;
00062         }
00063         int as_int(int def = 0) const {
00064             return (type == NUMBER) ? static_cast<int>(num_val) : def;
00065         }
00066     };
00067
00068     static JsonValue parse(const std::string& input) {
00069         MiniJsonParser p(input);
00070         return p.parse_value();
00071     }
00072 private:
00073     std::string_view src_;
00074     size_t pos_;
00075
00076     explicit MiniJsonParser(const std::string& s) : src_(s), pos_(0) {}
00077
00078     void skip_ws() {
00079         while (pos_ < src_.size() &&
00080             std::isspace(static_cast<unsigned char>(src_[pos_])))
00081             ++pos_;
00082     }
00083
00084     char peek() const {
00085         if (pos_ >= src_.size())
00086             throw std::runtime_error("MiniJsonParser: unexpected end of input");
00087         return src_[pos_];
00088     }
00089
00090     char advance() {
00091         if (pos_ >= src_.size())
00092             throw std::runtime_error("MiniJsonParser: unexpected end of input");
00093         return src_[pos_++];
00094     }
00095
00096     void expect(char c) {
00097         skip_ws();
00098         if (advance() != c)
00099             throw std::runtime_error(
00100                 std::string("MiniJsonParser: expected '" + c +
00101                     "' at position " + std::to_string(pos_ - 1));
00102     }
00103
00104     JsonValue parse_value() {
00105         skip_ws();
00106         char c = peek();
00107         if (c == '"') return parse_string_value();
00108         if (c == '{') return parse_object();
00109         if (c == '[') return parse_array();
00110         if (c == '-' || std::isdigit(static_cast<unsigned char>(c)))
00111             return parse_number();
00112         if (c == 't' || c == 'f') return parse_bool();
00113         if (c == 'n') return parse_null();
00114         throw std::runtime_error(
00115             std::string("MiniJsonParser: unexpected '" + c +
00116                 "' at position " + std::to_string(pos_));
00117     }
00118
00119     JsonValue parse_string_value() {
00120         JsonValue v;
00121         v.type = JsonValue::STRING;
00122         v.str_val = parse_string();
00123         return v;
00124     }
00125
00126     std::string parse_string() {
00127         expect('"');
00128         std::string result;
00129         while (pos_ < src_.size()) {

```

```

00128         char c = src_[pos_++];
00129         if (c == '"') return result;
00130         if (c == '\\') {
00131             if (pos_ >= src_.size())
00132                 throw std::runtime_error("MiniJsonParser: unterminated escape");
00133             char esc = src_[pos_++];
00134             switch (esc) {
00135                 case '"': result += '"'; break;
00136                 case '\\': result += '\\'; break;
00137                 case '/': result += '/'; break;
00138                 case 'n': result += '\n'; break;
00139                 case 't': result += '\t'; break;
00140                 case 'r': result += '\r'; break;
00141                 default: result += esc; break;
00142             }
00143         } else {
00144             result += c;
00145         }
00146     }
00147     throw std::runtime_error("MiniJsonParser: unterminated string");
00148 }
00149
00150 JsonValue parse_number() {
00151     JsonValue v;
00152     v.type = JsonValue::NUMBER;
00153     size_t start = pos_;
00154     if (src_[pos_] == '-') ++pos_;
00155     while (pos_ < src_.size() &&
00156           std::isdigit(static_cast<unsigned char>(src_[pos_])))
00157         ++pos_;
00158     if (pos_ < src_.size() && src_[pos_] == '.') {
00159         ++pos_;
00160         while (pos_ < src_.size() &&
00161               std::isdigit(static_cast<unsigned char>(src_[pos_])))
00162             ++pos_;
00163     }
00164     if (pos_ < src_.size() && (src_[pos_] == 'e' || src_[pos_] == 'E')) {
00165         ++pos_;
00166         if (pos_ < src_.size() && (src_[pos_] == '+' || src_[pos_] == '-'))
00167             ++pos_;
00168         while (pos_ < src_.size() &&
00169               std::isdigit(static_cast<unsigned char>(src_[pos_])))
00170             ++pos_;
00171     }
00172     std::string num_str(src_.substr(start, pos_ - start));
00173     v.num_val = std::stod(num_str);
00174     return v;
00175 }
00176
00177 JsonValue parse_object() {
00178     JsonValue v;
00179     v.type = JsonValue::OBJECT;
00180     expect('{');
00181     skip_ws();
00182     if (peek() == '}') { advance(); return v; }
00183     while (true) {
00184         skip_ws();
00185         std::string key = parse_string();
00186         skip_ws(); expect(':');
00187         JsonValue val = parse_value();
00188         v.obj_val.emplace_back(std::move(key), std::move(val));
00189         skip_ws();
00190         char c = advance();
00191         if (c == '}') break;
00192         if (c != ',')
00193             throw std::runtime_error(
00194                 "MiniJsonParser: expected ',' or '}' in object");
00195     }
00196     return v;
00197 }
00198
00199 JsonValue parse_array() {
00200     JsonValue v;
00201     v.type = JsonValue::ARRAY;
00202     expect '[';
00203     skip_ws();
00204     if (peek() == ']') { advance(); return v; }
00205     while (true) {
00206         v.arr_val.push_back(parse_value());
00207         skip_ws();
00208         char c = advance();
00209         if (c == ']') break;
00210         if (c != ',')
00211             throw std::runtime_error(
00212                 "MiniJsonParser: expected ',' or ']' in array");
00213     }
00214     return v;

```

```

00215     }
00216
00217     JsonValue parse_bool() {
00218         JsonValue v;
00219         v.type = JsonValue::NUMBER;
00220         if (src_.substr(pos_, 4) == "true") { pos_ += 4; v.num_val = 1.0; }
00221         else if (src_.substr(pos_, 5) == "false") { pos_ += 5; v.num_val = 0.0; }
00222         else throw std::runtime_error(
00223             "MiniJsonParser: unexpected token at " + std::to_string(pos_));
00224         return v;
00225     }
00226
00227     JsonValue parse_null() {
00228         if (src_.substr(pos_, 4) == "null") { pos_ += 4; return JsonValue{}; }
00229         throw std::runtime_error(
00230             "MiniJsonParser: unexpected token at " + std::to_string(pos_));
00231     }
00232 };
00233
00234 inline std::string read_file_to_string(const std::string& filepath) {
00235     std::ifstream ifs(filepath, std::ios::in | std::ios::binary);
00236     if (!ifs.is_open())
00237         throw std::runtime_error("Failed to open file: " + filepath);
00238     std::ostringstream oss;
00239     oss << ifs.rdbuf();
00240     return oss.str();
00241 }
00242
00243 // namespace detail
00244
00245 // =====
00246 // Rule loading from JSON
00247 // =====
00248
00249 inline std::vector<RuleDefinition> load_rules(const std::string& filepath) {
00250     std::string content = detail::read_file_to_string(filepath);
00251     auto root = detail::MiniJsonParser::parse(content);
00252
00253     if (root.type != detail::MiniJsonParser::JsonValue::ARRAY)
00254         throw std::runtime_error(
00255             "load_rules: expected top-level JSON array in " + filepath);
00256
00257     std::vector<RuleDefinition> rules;
00258     rules.reserve(root.arr_val.size());
00259
00260     for (const auto& elem : root.arr_val) {
00261         if (elem.type != detail::MiniJsonParser::JsonValue::OBJECT)
00262             throw std::runtime_error(
00263                 "load_rules: expected each element to be an object");
00264
00265         RuleDefinition rule;
00266
00267         const auto* rid = elem.find("rule_id");
00268         if (!rid) throw std::runtime_error("Rule missing 'rule_id' field");
00269         rule.rule_id = rid->as_string();
00270
00271         const auto* type_val = elem.find("type");
00272         if (!type_val) throw std::runtime_error("Rule missing 'type' field");
00273         rule.type = string_to_ruletype(type_val->as_string());
00274
00275         // Priority defaults to LOW if absent (spec says "optionally")
00276         const auto* prio = elem.find("priority");
00277         rule.priority = prio ? string_to_priority(prio->as_string())
00278             : Priority::LOW;
00279
00280         const auto* sensor = elem.find("sensor_id");
00281         if (sensor) rule.sensor_id = sensor->as_string();
00282
00283         const auto* op_val = elem.find("operator");
00284         if (op_val) rule.op = string_to_compop(op_val->as_string());
00285
00286         const auto* thresh = elem.find("value");
00287         if (thresh) rule.threshold = thresh->as_number();
00288
00289         const auto* consec = elem.find("consecutive_measurements");
00290         if (consec) rule.consecutive_measurements = consec->as_int(1);
00291
00292         const auto* logic_val = elem.find("logic");
00293         if (logic_val) rule.logic = logic_val->as_string();
00294
00295         const auto* sub = elem.find("conditions");
00296         if (!sub) sub = elem.find("sub_rules");
00297         if (sub && sub->type == detail::MiniJsonParser::JsonValue::ARRAY) {
00298             for (const auto& e : sub->arr_val)
00299                 rule.sub_rules.push_back(e.as_string());
00300         }
00301     }
00302 }

```

```

00311         rules.push_back(std::move(rule));
00312     }
00313
00314     return rules;
00315 }
00316
00317 // =====
00318 // Priority-based rule sorting
00319 // =====
00320
00321 inline void sort_by_priority(std::vector<const RuleDefinition*>& rules) {
00322     std::sort(rules.begin(), rules.end(),
00323         [](const RuleDefinition* a, const RuleDefinition* b) {
00324             return static_cast<int>(a->priority) >
00325                 static_cast<int>(b->priority);
00326         });
00327 }
00328
00329 // =====
00330 // Sensor token mapping
00331 // =====
00332
00333 inline void assign_sensor_tokens(
00334     std::vector<RuleDefinition*>& rules,
00335     const std::unordered_map<std::string, int>& token_map)
00336 {
00337     for (auto& r : rules) {
00338         if (r.type == RuleType::CORRELATION) {
00339             r.sensor_token_idx = -1;
00340             continue;
00341         }
00342         auto it = token_map.find(r.sensor_id);
00343         r.sensor_token_idx = (it != token_map.end()) ? it->second : -1;
00344     }
00345 }
00346
00347 // =====
00348 // Inline rule evaluators
00349 // =====
00350
00351 inline bool evaluate_threshold(double value,
00352                               const RuleDefinition& rule) noexcept {
00353     return evaluate_comparison(value, rule.op, rule.threshold);
00354 }
00355
00356 inline bool evaluate_step_diff(double value, const SensorState& state,
00357                               const RuleDefinition& rule) noexcept {
00358     if (!state.has_previous) return false; // First reading → nominal
00359     double delta = value - state.previous_value;
00360     return evaluate_comparison(delta, rule.op, rule.threshold);
00361 }
00362
00363 inline bool evaluate_stateful(double value, SensorState& state,
00364                               const RuleDefinition& rule,
00365                               int rule_slot) noexcept {
00366     bool violated = evaluate_comparison(value, rule.op, rule.threshold);
00367
00368     if (violated) {
00369         int& count = state.consecutive_violations[rule_slot];
00370         ++count;
00371         return (count >= rule.consecutive_measurements);
00372     } else {
00373         state.consecutive_violations[rule_slot] = 0;
00374         return false;
00375     }
00376 }
00377
00378 } // namespace astralog
00379
00380 #endif // ASTRALOG_RULES_ENGINE_HPP

```

8.10 src/timestamp_processor.hpp File Reference

Per-timestamp grouping, parallel rule evaluation, and correlation.

```

#include <vector>
#include <string>
#include <unordered_map>
#include <algorithm>

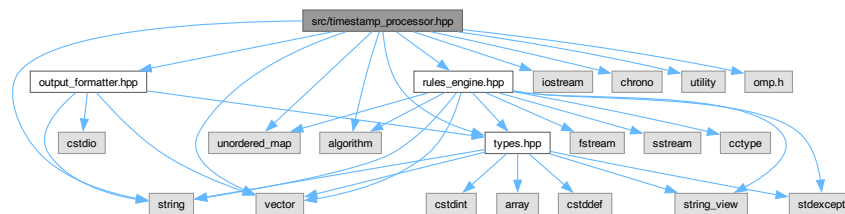
```

```

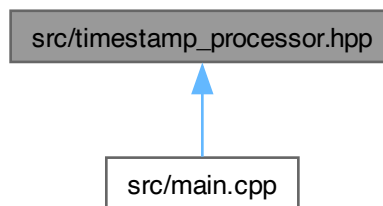
#include <iostream>
#include <chrono>
#include <utility>
#include <omp.h>
#include "types.hpp"
#include "rules_engine.hpp"
#include "output_formatter.hpp"

```

Include dependency graph for timestamp_processor.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- struct `astralog::SensorRuleSet`
Pre-built lookup table mapping a sensor token to its applicable rules, sorted by priority within each type.
- struct `astralog::SensorRuleSet::StatefulEntry`
- struct `astralog::PreparedRules`
Rule lookup structures built once after configuration loading.
- struct `astralog::PipelineState`
Wraps per-sensor `SensorState` objects so they persist across batch boundaries.
- struct `astralog::ProcessingResult`
Aggregated output of the full processing pipeline.

Namespaces

- namespace `astralog`

Functions

- `PreparedRules astralog::prepare_rules` (`std::vector< RuleDefinition > &all_rules`, `const std::unordered_map< std::string, int > &token_map`, `int num_sensors`)
Precompute all per-sensor and correlation rule lookup structures.
- `std::vector< TimestampGroup > astralog::build_timestamp_groups` (`const std::vector< ParsedRecord > &records`)
Group consecutive records by timestamp.
- `std::vector< std::vector< std::pair< size_t, size_t > > > astralog::build_per_sensor_sequences` (`const std::vector< TimestampGroup > &groups`, `int num_sensors`)
For each sensor, build an ordered sequence of (group_idx, reading_idx) pairs.
- `std::vector< std::vector< RuleViolation > > astralog::evaluate_rules_parallel` (`const std::vector< TimestampGroup > &groups`, `const std::vector< std::vector< std::pair< size_t, size_t > > > &sensor_sequences`, `const std::vector< SensorRuleSet > &sensor_rules`, `int num_sensors`, `PipelineState *pipeline`, `_state=nullptr`)
Evaluate threshold, step_diff, and stateful rules for all sensors in parallel using OpenMP.
- `void astralog::evaluate_correlations` (`const std::vector< TimestampGroup > &groups`, `std::vector< std::vector< RuleViolation > > ×tamp_violations`, `const std::vector< const RuleDefinition * > &correlation_rules`)
Evaluate correlation rules across all timestamps.
- `void astralog::sort_violations_for_output` (`std::vector< std::vector< RuleViolation > > ×tamp_violations`)
- `void astralog::generate_output_parallel` (`const std::vector< TimestampGroup > &groups`, `const std::vector< std::vector< RuleViolation > > ×tamp_violations`, `const std::vector< std::string > &inverse_token_map`, `std::vector< std::string > &valid_lines`, `std::vector< std::string > &alarm_lines`)
Generate output strings for all timestamps in parallel.
- `ProcessingResult astralog::process_pipeline` (`const std::vector< ParsedRecord > &records`, `const PreparedRules &prepared_rules`, `const std::vector< std::string > &inverse_token_map`, `PipelineState &state`, `bool verbose=true`)
Run the full processing pipeline: group → evaluate → correlate → output.

8.10.1 Detailed Description

Per-timestamp grouping, parallel rule evaluation, and correlation.

8.10.1.1 Processing Pipeline

Phase 1 (Sequential): Build timestamp groups Scan the ordered record list and group consecutive records that share the same timestamp into `TimestampGroup` objects.

Phase 2 (Sequential): Build per-sensor reading sequences For each sensor, build an ordered list of (group_idx, reading_idx) pairs for O(1) lookup during evaluation.

Phase 3 (Parallel): Per-sensor rule evaluation OpenMP parallel-for across sensors. Each thread processes one sensor's entire timeline, evaluating threshold, step_diff, and stateful rules. This exploits the fact that different sensors are fully independent. Step_diff and stateful rules require per-sensor sequential processing, which is naturally satisfied within each thread.

Phase 4 (Sequential): Merge + Correlation evaluation Merge per-thread violation maps. Then walk timestamps sequentially to evaluate correlation rules (AND/OR of sub-rule results).

Phase 5 (Parallel): Output formatting OpenMP parallel-for across timestamps. Each thread formats its assigned timestamp as either a NOMINAL or ANOMALOUS line.

8.10.1.2 Parallelism Characteristics

- Phase 1-2: $O(N)$ sequential scans, fast
- Phase 3: parallelism number of sensors (12 in current config)
- Phase 4: sequential but lightweight (correlation rules only)
- Phase 5: parallelism number of timestamps (thousands+)

8.11 timestamp_processor.hpp

[Go to the documentation of this file.](#)

```

00001
00037
00038 #ifndef ASTRALOG_TIMESTAMP_PROCESSOR_HPP
00039 #define ASTRALOG_TIMESTAMP_PROCESSOR_HPP
00040
00041 #include <vector>
00042 #include <string>
00043 #include <unordered_map>
00044 #include <algorithm>
00045 #include <iostream>
00046 #include <chrono>
00047 #include <utility>
00048
00049 #include <omp.h>
00050
00051 #include "types.hpp"
00052 #include "rules_engine.hpp"
00053 #include "output_formatter.hpp"
00054
00055 namespace astralog {
00056
00057 // =====
00058 // Per-sensor rule lookup table
00059 // =====
00060
00066 struct SensorRuleSet {
00067     std::vector<const RuleDefinition*> threshold;
00068     std::vector<const RuleDefinition*> step_diff;
00069
00070     struct StatefulEntry {
00071         const RuleDefinition* rule;
00072         int slot;
00073     };
00074     std::vector<StatefulEntry> stateful;
00075 };
00076
00084 struct PreparedRules {
00085     std::vector<SensorRuleSet> sensor_rules;
00086     std::vector<const RuleDefinition*> correlation_rules;
00087     size_t threshold_count = 0;
00088     size_t step_diff_count = 0;
00089     size_t stateful_count = 0;
00090 };
00091
00092 // =====
00093 // PipelineState: cross-batch sensor state persistence
00094 // =====
00095
00111 struct PipelineState {
00112     std::vector<SensorState> sensor_states;
00113     bool initialized = false;
00114
00122     void initialize(int num_sensors,
00123                    const std::vector<SensorRuleSet*>& sensor_rules) {
00124         if (initialized) return;
00125         sensor_states.resize(num_sensors);
00126         for (int s = 0; s < num_sensors; ++s) {
00127             sensor_states[s].consecutive_violations.assign(
00128                 sensor_rules[s].stateful.size(), 0);
00129         }
00130         initialized = true;
00131     }
00132 };
00133

```

```

00134 // =====
00135 // Rule preparation
00136 // =====
00137
00144 inline PreparedRules prepare_rules(
00145     std::vector<RuleDefinition*> & all_rules,
00146     const std::unordered_map<std::string, int> & token_map,
00147     int num_sensors)
00148 {
00149     assign_sensor_tokens(all_rules, token_map);
00150
00151     std::vector<const RuleDefinition*> threshold_rules;
00152     std::vector<const RuleDefinition*> step_diff_rules;
00153     std::vector<const RuleDefinition*> stateful_rules;
00154     std::vector<const RuleDefinition*> correlation_rules;
00155
00156     threshold_rules.reserve(all_rules.size());
00157     step_diff_rules.reserve(all_rules.size());
00158     stateful_rules.reserve(all_rules.size());
00159     correlation_rules.reserve(all_rules.size());
00160
00161     for (const auto& r : all_rules) {
00162         switch (r.type) {
00163             case RuleType::THRESHOLD: threshold_rules.push_back(&r); break;
00164             case RuleType::STEP_DIFF: step_diff_rules.push_back(&r); break;
00165             case RuleType::STATEFUL: stateful_rules.push_back(&r); break;
00166             case RuleType::CORRELATION: correlation_rules.push_back(&r); break;
00167         }
00168     }
00169
00170     sort_by_priority(threshold_rules);
00171     sort_by_priority(step_diff_rules);
00172     sort_by_priority(stateful_rules);
00173     sort_by_priority(correlation_rules);
00174
00175     PreparedRules prepared;
00176     prepared.threshold_count = threshold_rules.size();
00177     prepared.step_diff_count = step_diff_rules.size();
00178     prepared.stateful_count = stateful_rules.size();
00179     prepared.correlation_rules = std::move(correlation_rules);
00180     prepared.sensor_rules.resize(num_sensors);
00181
00182     for (const auto* r : threshold_rules) {
00183         if (r->sensor_token_idx >= 0 && r->sensor_token_idx < num_sensors)
00184             prepared.sensor_rules[r->sensor_token_idx].threshold.push_back(r);
00185     }
00186     for (const auto* r : step_diff_rules) {
00187         if (r->sensor_token_idx >= 0 && r->sensor_token_idx < num_sensors)
00188             prepared.sensor_rules[r->sensor_token_idx].step_diff.push_back(r);
00189     }
00190     for (const auto* r : stateful_rules) {
00191         if (r->sensor_token_idx >= 0 && r->sensor_token_idx < num_sensors) {
00192             auto& stateful = prepared.sensor_rules[r->sensor_token_idx].stateful;
00193             int slot = static_cast<int>(stateful.size());
00194             stateful.push_back({r, slot});
00195         }
00196     }
00197
00198     return prepared;
00199 }
00200
00201 // =====
00202 // Phase 1: Build timestamp groups
00203 // =====
00204
00215 inline std::vector<TimestampGroup> build_timestamp_groups(
00216     const std::vector<ParsedRecord> & records)
00217 {
00218     std::vector<TimestampGroup> groups;
00219     if (records.empty()) return groups;
00220
00221     // Better estimate: ~12 sensors per timestamp in typical satellite data
00222     const size_t estimated_groups = records.size() / 12 + 1;
00223     groups.reserve(estimated_groups);
00224
00225     std::string current_ts;
00226     for (const auto& rec : records) {
00227         if (rec.timestamp != current_ts) {
00228             groups.push_back({});
00229             groups.back().timestamp = rec.timestamp;
00230             groups.back().readings.reserve(16); // Typical sensor count
00231             current_ts = rec.timestamp;
00232         }
00233
00234         groups.back().readings.push_back({rec.sensor_token, rec.value});
00235     }
00236

```

```

00237     return groups;
00238 }
00239
00240 // =====
00241 // Phase 2: Build per-sensor reading sequences
00242 // =====
00243
00256 inline std::vector<std::vector<std::pair<size_t, size_t>>
00257 build_per_sensor_sequences(const std::vector<TimestampGroup>& groups,
00258                             int num_sensors)
00259 {
00260     std::vector<std::vector<std::pair<size_t, size_t>> sequences(num_sensors);
00261
00262     // Pre-allocate: each sensor appears roughly once per timestamp, with
00263     // slack for slightly out-of-sync or duplicated sensor packets.
00264     const size_t reserve_per_sensor = groups.size() + groups.size() / 2 + 1;
00265     for (auto& seq : sequences) {
00266         seq.reserve(reserve_per_sensor);
00267     }
00268
00269     for (size_t g = 0; g < groups.size(); ++g) {
00270         for (size_t r = 0; r < groups[g].readings.size(); ++r) {
00271             int token = groups[g].readings[r].sensor_token;
00272             if (token >= 0 && token < num_sensors) {
00273                 sequences[token].push_back({g, r});
00274             }
00275         }
00276     }
00277
00278     return sequences;
00279 }
00280
00281 // =====
00282 // Phase 3: Parallel per-sensor rule evaluation
00283 // =====
00284
00303 inline std::vector<std::vector<RuleViolation>> evaluate_rules_parallel(
00304     const std::vector<TimestampGroup>& groups,
00305     const std::vector<std::vector<std::pair<size_t, size_t>>& sensor_sequences,
00306     const std::vector<SensorRuleSet>& sensor_rules,
00307     int num_sensors,
00308     PipelineState* pipeline_state = nullptr)
00309 {
00310     const int num_threads = omp_get_max_threads();
00311     const size_t num_groups = groups.size();
00312
00313     // Per-thread violation storage: flat pre-allocated 2D vector.
00314     // thread_violations[tid][group_idx] is a vector of violations.
00315     // This replaces the old unordered_map, eliminating all hashing overhead.
00316     std::vector<std::vector<std::vector<RuleViolation>>
00317         thread_violations(num_threads,
00318                             std::vector<std::vector<RuleViolation>>(num_groups));
00319
00320     #pragma omp parallel
00321     {
00322         int tid = omp_get_thread_num();
00323         auto& my_violations = thread_violations[tid];
00324
00325         auto append_violation = [&](size_t group_idx,
00326                                     const RuleDefinition& rule,
00327                                     int sensor_token,
00328                                     double value) {
00329             auto& group_violations = my_violations[group_idx];
00330             if (group_violations.empty()) {
00331                 group_violations.reserve(8);
00332             }
00333             RuleViolation violation(&rule);
00334             violation.add(sensor_token, value);
00335             group_violations.push_back(std::move(violation));
00336         };
00337
00338         // Each sensor is processed independently → parallel for across sensors
00339         #pragma omp for schedule(dynamic)
00340         for (int s = 0; s < num_sensors; ++s) {
00341             const auto& seq = sensor_sequences[s];
00342             if (seq.empty()) continue;
00343
00344             const auto& rules = sensor_rules[s];
00345
00346             // Per-sensor state: use existing state from PipelineState if
00347             // available (cross-batch persistence), otherwise create fresh.
00348             SensorState state;
00349             if (pipeline_state && pipeline_state->initialized) {
00350                 state = pipeline_state->sensor_states[s];
00351             } else {
00352                 state.consecutive_violations.assign(rules.stateful.size(), 0);
00353             }

```

```

00354
00355     for (const auto& [group_idx, reading_idx] : seq) {
00356         const auto& reading = groups[group_idx].readings[reading_idx];
00357         double value = reading.value;
00358
00359         // --- THRESHOLD rules (sorted by priority) ---
00360         for (const auto* rule : rules.threshold) {
00361             if (evaluate_threshold(value, *rule)) {
00362                 append_violation(group_idx, *rule,
00363                                 reading.sensor_token, value);
00364             }
00365         }
00366
00367         // --- STEP_DIFF rules (signed delta, no abs!) ---
00368         if (state.has_previous) {
00369             for (const auto* rule : rules.step_diff) {
00370                 if (evaluate_step_diff(value, state, *rule)) {
00371                     append_violation(group_idx, *rule,
00372                                     reading.sensor_token, value);
00373                 }
00374             }
00375         }
00376
00377         // --- STATEFUL rules ---
00378         for (size_t si = 0; si < rules.stateful.size(); ++si) {
00379             const auto& entry = rules.stateful[si];
00380             if (evaluate_stateful(value, state, *entry.rule,
00381                                 entry.slot)) {
00382                 append_violation(group_idx, *entry.rule,
00383                                 reading.sensor_token, value);
00384             }
00385         }
00386
00387         // Update sensor state for next iteration
00388         state.previous_value = value;
00389         state.has_previous = true;
00390     }
00391
00392     // Write back final sensor state for cross-batch persistence
00393     if (pipeline_state) {
00394         pipeline_state->sensor_states[s] = state;
00395     }
00396 }
00397 }
00398
00399 // Merge per-thread violations into per-timestamp vector.
00400 // O(1) array access replaces the old hash map iteration.
00401 std::vector<std::vector<RuleViolation>> result(num_groups);
00402 for (int t = 0; t < num_threads; ++t) {
00403     for (size_t g = 0; g < num_groups; ++g) {
00404         auto& src = thread_violations[t][g];
00405         if (!src.empty()) {
00406             auto& dst = result[g];
00407             dst.insert(dst.end(),
00408                       std::make_move_iterator(src.begin()),
00409                       std::make_move_iterator(src.end()));
00410         }
00411     }
00412 }
00413
00414 return result;
00415 }
00416
00417 // =====
00418 // Phase 4: Correlation evaluation
00419 // =====
00420
00421 inline void evaluate_correlations(
00422     const std::vector<TimestampGroup>& groups,
00423     std::vector<std::vector<RuleViolation>>& timestamp_violations,
00424     const std::vector<const RuleDefinition*>& correlation_rules)
00425 {
00426     if (correlation_rules.empty()) return;
00427
00428     // Small struct for linear-scan lookup (replaces unordered_map).
00429     // Violations per timestamp are typically 1-5, so linear scan is
00430     // faster than hashing for these tiny N values.
00431     struct FiredEntry {
00432         const RuleDefinition* rule; // non-owning pointer, avoids copy
00433         int sensor_token;
00434         double value;
00435     };
00436
00437     for (size_t g = 0; g < groups.size(); ++g) {
00438         if (timestamp_violations[g].empty()) continue;
00439         timestamp_violations[g].reserve(

```

```

00454         timestamp_violations[g].size() + correlation_rules.size());
00455
00456     // Build flat lookup array of fired rule IDs and their data
00457     std::vector<FiredEntry> fired;
00458     fired.reserve(timestamp_violations[g].size());
00459
00460     for (const auto& v : timestamp_violations[g]) {
00461         int st = v.empty() ? -1 : v.sensor_token_at(0);
00462         double val = v.empty() ? 0.0 : v.value_at(0);
00463         fired.push_back({v.rule, st, val});
00464     }
00465
00466     // Lambda: linear scan to check if a rule_id fired
00467     auto find_fired = [&](const std::string& target) -> const FiredEntry* {
00468         for (const auto& f : fired) {
00469             if (f.rule && f.rule->rule_id == target) return &f;
00470         }
00471         return nullptr;
00472     };
00473
00474     // Evaluate each correlation rule
00475     for (const auto* rule : correlation_rules) {
00476         bool triggered = false;
00477
00478         if (rule->logic == "AND") {
00479             triggered = true;
00480             for (const auto& sr_id : rule->sub_rules) {
00481                 if (!find_fired(sr_id)) {
00482                     triggered = false;
00483                     break;
00484                 }
00485             }
00486         } else if (rule->logic == "OR") {
00487             triggered = false;
00488             for (const auto& sr_id : rule->sub_rules) {
00489                 if (find_fired(sr_id)) {
00490                     triggered = true;
00491                     break;
00492                 }
00493             }
00494         }
00495
00496         if (triggered) {
00497             RuleViolation corr_v(rule);
00498
00499             // Collect sensors and values from all sub-rules
00500             for (const auto& sr_id : rule->sub_rules) {
00501                 const FiredEntry* entry = find_fired(sr_id);
00502                 if (entry && entry->sensor_token >= 0) {
00503                     corr_v.add(entry->sensor_token, entry->value);
00504                 }
00505             }
00506
00507             timestamp_violations[g].push_back(std::move(corr_v));
00508         }
00509     }
00510 }
00511
00512 inline void sort_violations_for_output(
00513     std::vector<std::vector<RuleViolation>>& timestamp_violations)
00514 {
00515     auto priority_rank = [](const RuleViolation& v) {
00516         return v.rule ? static_cast<int>(v.rule->priority) : -1;
00517     };
00518
00519     auto rule_id = [](const RuleViolation& v) -> const std::string& {
00520         static const std::string unknown = "UNKNOWN_RULE";
00521         return v.rule ? v.rule->rule_id : unknown;
00522     };
00523
00524     for (auto& violations : timestamp_violations) {
00525         std::stable_sort(
00526             violations.begin(), violations.end(),
00527             [&](const RuleViolation& a, const RuleViolation& b) {
00528                 int pa = priority_rank(a);
00529                 int pb = priority_rank(b);
00530                 if (pa != pb) return pa > pb; // HIGH > MEDIUM > LOW
00531                 return rule_id(a) < rule_id(b);
00532             });
00533     }
00534 }
00535
00536 // =====
00537 // Phase 5: Parallel output generation
00538 // =====
00539
00540

```

```

00553 inline void generate_output_parallel(
00554     const std::vector<TimestampGroup>& groups,
00555     const std::vector<std::vector<RuleViolation>>& timestamp_violations,
00556     const std::vector<std::string>& inverse_token_map,
00557     std::vector<std::string>& valid_lines,
00558     std::vector<std::string>& alarm_lines)
00559 {
00560     const size_t n = groups.size();
00561     valid_lines.resize(n);
00562     alarm_lines.resize(n);
00563
00564     #pragma omp parallel for schedule(static)
00565     for (size_t g = 0; g < n; ++g) {
00566         if (timestamp_violations[g].empty()) {
00567             // NOMINAL: aggregated line with all sensor values
00568             valid_lines[g] = format_nominal_line(groups[g], inverse_token_map);
00569         } else {
00570             // ANOMALOUS: one line per violated rule
00571             alarm_lines[g] = format_alarm_lines(groups[g].timestamp,
00572                                                 timestamp_violations[g],
00573                                                 inverse_token_map);
00574         }
00575     }
00576 }
00577
00578 // =====
00579 // Convenience: Full processing pipeline
00580 // =====
00581
00582 struct ProcessingResult {
00583     std::vector<std::string> valid_lines;
00584     std::vector<std::string> alarm_lines;
00585     uint64_t num_timestamps;
00586     uint64_t num_anomalies;
00587     uint64_t num_nominal;
00588     double eval_ms;
00589     double total_ms;
00590 };
00591
00592 inline ProcessingResult process_pipeline(
00593     const std::vector<ParsedRecord>& records,
00594     const PreparedRules& prepared_rules,
00595     const std::vector<std::string>& inverse_token_map,
00596     PipelineState& state,
00597     bool verbose = true)
00598 {
00599     auto pipeline_start = std::chrono::high_resolution_clock::now();
00600     const int num_sensors = static_cast<int>(prepared_rules.sensor_rules.size());
00601
00602     // Initialize PipelineState on first batch
00603     state.initialize(num_sensors, prepared_rules.sensor_rules);
00604
00605     // --- Phase 1: Build timestamp groups ---
00606     auto phase1_start = std::chrono::high_resolution_clock::now();
00607     auto groups = build_timestamp_groups(records);
00608     auto phase1_end = std::chrono::high_resolution_clock::now();
00609     double phase1_ms = std::chrono::duration<double, std::milli>(
00610         phase1_end - phase1_start).count();
00611
00612     if (verbose) {
00613         std::cerr << "[pipeline] Phase 1: Grouped " << groups.size()
00614             << " unique timestamps in " << phase1_ms << " ms\n";
00615     }
00616
00617     // --- Phase 2: Build per-sensor reading sequences ---
00618     auto phase2_start = std::chrono::high_resolution_clock::now();
00619     auto sensor_sequences = build_per_sensor_sequences(groups, num_sensors);
00620     auto phase2_end = std::chrono::high_resolution_clock::now();
00621     double phase2_ms = std::chrono::duration<double, std::milli>(
00622         phase2_end - phase2_start).count();
00623
00624     if (verbose) {
00625         std::cerr << "[pipeline] Phase 2: Built per-sensor sequences in "
00626             << phase2_ms << " ms\n";
00627     }
00628
00629     // --- Phase 3: Parallel per-sensor rule evaluation ---
00630     auto phase3_start = std::chrono::high_resolution_clock::now();
00631     auto timestamp_violations = evaluate_rules_parallel(
00632         groups, sensor_sequences, prepared_rules.sensor_rules,
00633         num_sensors, &state);
00634     auto phase3_end = std::chrono::high_resolution_clock::now();
00635     double phase3_ms = std::chrono::duration<double, std::milli>(
00636         phase3_end - phase3_start).count();
00637
00638     if (verbose) {
00639         std::cerr << "[pipeline] Phase 3: Parallel rule evaluation in "

```

```

00659         « phase3_ms « " ms (" « omp_get_max_threads()
00660         « " threads)\n";
00661     }
00662
00663     // --- Phase 4: Correlation evaluation ---
00664     auto phase4_start = std::chrono::high_resolution_clock::now();
00665     evaluate_correlations(groups, timestamp_violations,
00666                         prepared_rules.correlation_rules);
00667     sort_violations_for_output(timestamp_violations);
00668     auto phase4_end = std::chrono::high_resolution_clock::now();
00669     double phase4_ms = std::chrono::duration<double, std::milli>(
00670         phase4_end - phase4_start).count();
00671
00672     if (verbose) {
00673         std::cerr « "[pipeline] Phase 4: Correlation evaluation in "
00674                 « phase4_ms « " ms\n";
00675     }
00676
00677     // --- Phase 5: Parallel output generation ---
00678     auto phase5_start = std::chrono::high_resolution_clock::now();
00679     std::vector<std::string> valid_lines, alarm_lines;
00680     generate_output_parallel(groups, timestamp_violations,
00681                             inverse_token_map,
00682                             valid_lines, alarm_lines);
00683     auto phase5_end = std::chrono::high_resolution_clock::now();
00684     double phase5_ms = std::chrono::duration<double, std::milli>(
00685         phase5_end - phase5_start).count();
00686
00687     if (verbose) {
00688         std::cerr « "[pipeline] Phase 5: Parallel output formatting in "
00689                 « phase5_ms « " ms\n";
00690     }
00691
00692     // --- Statistics ---
00693     uint64_t num_anomalies = 0;
00694     for (size_t g = 0; g < groups.size(); ++g) {
00695         if (!timestamp_violations[g].empty()) ++num_anomalies;
00696     }
00697
00698     auto pipeline_end = std::chrono::high_resolution_clock::now();
00699     double total_ms = std::chrono::duration<double, std::milli>(
00700         pipeline_end - pipeline_start).count();
00701
00702     ProcessingResult result;
00703     result.valid_lines = std::move(valid_lines);
00704     result.alarm_lines = std::move(alarm_lines);
00705     result.num_timestamps = groups.size();
00706     result.num_anomalies = num_anomalies;
00707     result.num_nominal = groups.size() - num_anomalies;
00708     result.eval_ms = phase3_ms;
00709     result.total_ms = total_ms;
00710
00711     return result;
00712 }
00713
00714 } // namespace astralog
00715
00716 #endif // ASTRALOG_TIMESTAMP_PROCESSOR_HPP

```

8.12 src/types.hpp File Reference

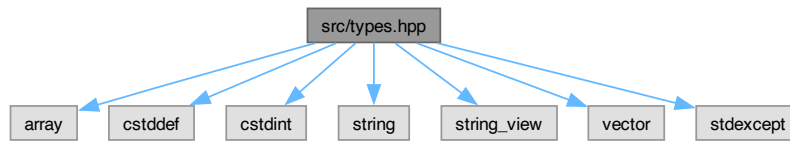
Core types, enumerations, and data structures for AstraLog-HPC.

```

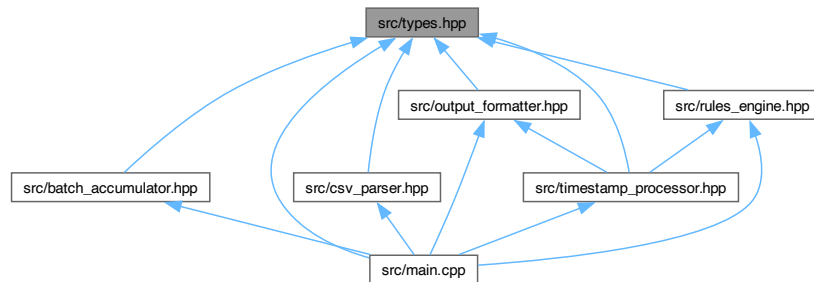
#include <array>
#include <cstdint>
#include <string>
#include <string_view>
#include <vector>
#include <stdexcept>

```

Include dependency graph for types.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- struct [astralog::RuleDefinition](#)
In-memory representation of a single monitoring rule loaded from JSON.
- struct [astralog::SensorState](#)
Per-sensor mutable state for sequential rule evaluation.
- struct [astralog::ParsedRecord](#)
- struct [astralog::TimestampGroup](#)
Groups all valid sensor readings that share the same timestamp.
- struct [astralog::TimestampGroup::Reading](#)
- struct [astralog::RuleViolation](#)
Represents a single rule violation at a given timestamp.

Namespaces

- namespace [astralog](#)

Enumerations

- enum class [astralog::Priority](#) : `uint8_t` { [astralog::LOW](#) = 0 , [astralog::MEDIUM](#) = 1 , [astralog::HIGH](#) = 2 }
Alert priority levels (LOW < MEDIUM < HIGH).
- enum class [astralog::RuleType](#) : `uint8_t` { [astralog::THRESHOLD](#) = 0 , [astralog::STEP_DIFF](#) = 1 , [astralog::STATEFUL](#) = 2 , [astralog::CORRELATION](#) = 3 }
Monitoring rule types.
- enum class [astralog::CompOp](#) : `uint8_t` { [astralog::GT](#) , [astralog::GE](#) , [astralog::LT](#) , [astralog::LE](#) , [astralog::EQ](#) , [astralog::NE](#) }
Comparison operators for rule evaluation.

Functions

- const char * `astralog::priority_to_string` (Priority p) noexcept
- Priority `astralog::string_to_priority` (std::string_view s)
- const char * `astralog::ruletype_to_string` (RuleType t) noexcept
- RuleType `astralog::string_to_ruletype` (std::string_view s)
- CompOp `astralog::string_to_compop` (std::string_view s)
- bool `astralog::evaluate_comparison` (double lhs, CompOp op, double rhs) noexcept
Inline comparison: lhs <op> rhs.

8.12.1 Detailed Description

Core types, enumerations, and data structures for AstraLog-HPC.

This header defines all shared types used across the processing pipeline:

- Priority, RuleType, CompOp enumerations with string conversion
- RuleDefinition: in-memory representation of a monitoring rule
- SensorState: per-sensor mutable state for step-diff and stateful rules
- ParsedRecord: a single validated CSV row
- TimestampGroup: all sensor readings at a single timestamp
- RuleViolation: a detected rule violation (simple or correlation)

8.13 types.hpp

[Go to the documentation of this file.](#)

```

00001
00013
00014 #ifndef ASTRALOG_TYPES_HPP
00015 #define ASTRALOG_TYPES_HPP
00016
00017 #include <array>
00018 #include <cstdint>
00019 #include <string>
00020 #include <string_view>
00021 #include <vector>
00022 #include <stdexcept>
00023
00024 namespace astralog {
00025
00026 // =====
00027 // Enumerations
00028 // =====
00029
00030 enum class Priority : uint8_t {
00031     LOW = 0,
00032     MEDIUM = 1,
00033     HIGH = 2
00034 };
00035
00036 inline const char* priority_to_string(Priority p) noexcept {
00037     switch (p) {
00038         case Priority::LOW: return "LOW";
00039         case Priority::MEDIUM: return "MEDIUM";
00040         case Priority::HIGH: return "HIGH";
00041         default: return "UNKNOWN";
00042     }
00043 }
00044
00045 inline Priority string_to_priority(std::string_view s) {

```

```

00048     if (s == "LOW" || s == "low") return Priority::LOW;
00049     if (s == "MEDIUM" || s == "medium") return Priority::MEDIUM;
00050     if (s == "HIGH" || s == "high") return Priority::HIGH;
00051     throw std::invalid_argument(std::string("Unknown priority: ") + std::string(s));
00052 }
00053
00055 enum class RuleType : uint8_t {
00056     THRESHOLD = 0,
00057     STEP_DIFF = 1,
00058     STATEFUL = 2,
00059     CORRELATION = 3
00060 };
00061
00062 inline const char* ruletype_to_string(RuleType t) noexcept {
00063     switch (t) {
00064         case RuleType::THRESHOLD: return "threshold";
00065         case RuleType::STEP_DIFF: return "step_diff";
00066         case RuleType::STATEFUL: return "stateful";
00067         case RuleType::CORRELATION: return "correlation";
00068         default: return "unknown";
00069     }
00070 }
00071
00072 inline RuleType string_to_ruletype(std::string_view s) {
00073     if (s == "threshold" || s == "simple") return RuleType::THRESHOLD;
00074     if (s == "step_difference" || s == "step_diff") return RuleType::STEP_DIFF;
00075     if (s == "stateful") return RuleType::STATEFUL;
00076     if (s == "correlation") return RuleType::CORRELATION;
00077     throw std::invalid_argument(std::string("Unknown rule type: ") + std::string(s));
00078 }
00079
00081 enum class CompOp : uint8_t {
00082     GT, GE, LT, LE, EQ, NE
00083 };
00084
00085 inline CompOp string_to_compop(std::string_view s) {
00086     if (s == ">") return CompOp::GT;
00087     if (s == ">=") return CompOp::GE;
00088     if (s == "<") return CompOp::LT;
00089     if (s == "<=") return CompOp::LE;
00090     if (s == "==") return CompOp::EQ;
00091     if (s == "!=") return CompOp::NE;
00092     throw std::invalid_argument(std::string("Unknown operator: ") + std::string(s));
00093 }
00094
00096 inline bool evaluate_comparison(double lhs, CompOp op, double rhs) noexcept {
00097     switch (op) {
00098         case CompOp::GT: return lhs > rhs;
00099         case CompOp::GE: return lhs >= rhs;
00100         case CompOp::LT: return lhs < rhs;
00101         case CompOp::LE: return lhs <= rhs;
00102         case CompOp::EQ: return lhs == rhs;
00103         case CompOp::NE: return lhs != rhs;
00104         default: return false;
00105     }
00106 }
00107
00108 // =====
00109 // Rule Definition
00110 // =====
00111
00116 struct RuleDefinition {
00117     std::string rule_id;
00118     RuleType type;
00119     std::string sensor_id;
00120     CompOp op;
00121     double threshold;
00122     Priority priority;
00123
00124     int consecutive_measurements;
00125
00126     // Correlation-specific fields
00127     std::string logic;
00128     std::vector<std::string> sub_rules;
00129
00130     // Runtime index: maps sensor_id → integer token for O(1) array access
00131     int sensor_token_idx;
00132
00133     RuleDefinition()
00134         : type(RuleType::THRESHOLD)
00135         , op(CompOp::GT)
00136         , threshold(0.0)
00137         , priority(Priority::LOW)
00138         , consecutive_measurements(1)
00139         , sensor_token_idx(-1)
00140     {}
00141 };

```

```

00142
00143 // =====
00144 // Sensor State (per-sensor mutable state for step-diff and stateful rules)
00145 // =====
00146
00154 struct alignas(64) SensorState {
00155     double           previous_value;
00156     bool             has_previous;
00157     std::vector<int> consecutive_violations;
00158
00159     SensorState() : previous_value(0.0), has_previous(false) {}
00160 };
00161
00162 // =====
00163 // Parsed Record (a single validated CSV row)
00164 // =====
00165
00166 struct ParsedRecord {
00167     std::string timestamp;
00168     double      value;
00169     int         sensor_token;
00170
00171     ParsedRecord() : value(0.0), sensor_token(-1) {}
00172 };
00173
00174 // =====
00175 // Timestamp Group (all sensor readings at a single timestamp)
00176 // =====
00177
00184 struct TimestampGroup {
00185     std::string timestamp;
00186
00187     struct Reading {
00188         int         sensor_token;
00189         double      value;
00190     };
00191     std::vector<Reading> readings;
00192 };
00193
00194 // =====
00195 // Rule Violation (detected anomaly -- simple or correlation)
00196 // =====
00197
00208 struct RuleViolation {
00209     static constexpr size_t INLINE_CAPACITY = 4;
00210
00211     const RuleDefinition* rule = nullptr;
00212     size_t count = 0;
00213
00214     std::array<int, INLINE_CAPACITY> sensor_tokens_inline{};
00215     std::array<double, INLINE_CAPACITY> values_inline{};
00216     std::vector<int> sensor_tokens_extra;
00217     std::vector<double> values_extra;
00218
00219     RuleViolation() = default;
00220     explicit RuleViolation(const RuleDefinition* rule_ptr) : rule(rule_ptr) {}
00221
00222     void add(int sensor_token, double value) {
00223         if (count < INLINE_CAPACITY) {
00224             sensor_tokens_inline[count] = sensor_token;
00225             values_inline[count] = value;
00226         } else {
00227             sensor_tokens_extra.push_back(sensor_token);
00228             values_extra.push_back(value);
00229         }
00230         ++count;
00231     }
00232
00233     size_t size() const noexcept {
00234         return count;
00235     }
00236
00237     bool empty() const noexcept {
00238         return count == 0;
00239     }
00240
00241     int sensor_token_at(size_t idx) const {
00242         return idx < INLINE_CAPACITY
00243             ? sensor_tokens_inline[idx]
00244             : sensor_tokens_extra[idx - INLINE_CAPACITY];
00245     }
00246
00247     double value_at(size_t idx) const {
00248         return idx < INLINE_CAPACITY
00249             ? values_inline[idx]
00250             : values_extra[idx - INLINE_CAPACITY];
00251     }

```

```

00252 };
00253
00254 } // namespace astralog
00255
00256 #endif // ASTRALOG_TYPES_HPP

```

8.14 src/yaml_parser.hpp File Reference

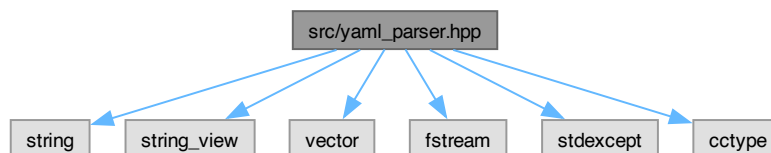
Lightweight YAML parser for sensor configuration files.

```

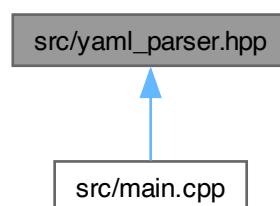
#include <string>
#include <string_view>
#include <vector>
#include <fstream>
#include <stdexcept>
#include <cctype>

```

Include dependency graph for yaml_parser.hpp:



This graph shows which files directly or indirectly include this file:



Namespaces

- namespace [astralog](#)

Functions

- `std::vector< std::string > astralog::load\_sensors\_from\_yaml (const std::string &filepath)`
Parse a YAML sensor configuration file and extract all sensor IDs.

8.14.1 Detailed Description

Lightweight YAML parser for sensor configuration files.

Implements a simple line-based parser that extracts sensor IDs from the sensors.yaml file without requiring external YAML libraries. Looks for lines containing "- id:" or "id:" patterns.

8.15 yaml_parser.hpp

[Go to the documentation of this file.](#)

```

00001
00009
00010 #ifndef ASTRALOG_YAML_PARSER_HPP
00011 #define ASTRALOG_YAML_PARSER_HPP
00012
00013 #include <string>
00014 #include <string_view>
00015 #include <vector>
00016 #include <fstream>
00017 #include <stdexcept>
00018 #include <cctype>
00019
00020 namespace astralog {
00021
00029 inline std::vector<std::string> load_sensors_from_yaml(const std::string& filepath) {
00030     std::ifstream ifs(filepath);
00031     if (!ifs.is_open()) {
00032         throw std::runtime_error("Failed to open YAML file: " + filepath);
00033     }
00034
00035     std::vector<std::string> sensor_ids;
00036     std::string line;
00037
00038     while (std::getline(ifs, line)) {
00039         // Strip leading whitespace
00040         size_t start = line.find_first_not_of(" \t\r\n");
00041         if (start == std::string::npos) continue;
00042         std::string_view trimmed(line.c_str() + start, line.size() - start);
00043
00044         // Strip trailing whitespace
00045         while (!trimmed.empty() &&
00046             std::isspace(static_cast<unsigned char>(trimmed.back()))) {
00047             trimmed.remove_suffix(1);
00048         }
00049
00050         // Match "- id: VALUE" or "id: VALUE"
00051         std::string_view value_part;
00052         if (trimmed.substr(0, 5) == "- id:") {
00053             value_part = trimmed.substr(5);
00054         } else if (trimmed.substr(0, 3) == "id:") {
00055             value_part = trimmed.substr(3);
00056         } else {
00057             continue;
00058         }
00059
00060         // Trim whitespace and quotes from the value
00061         size_t val_start = value_part.find_first_not_of(" \t'\"");
00062         if (val_start == std::string_view::npos) continue;
00063         std::string_view val = value_part.substr(val_start);
00064         while (!val.empty() &&
00065             (std::isspace(static_cast<unsigned char>(val.back())) ||
00066              val.back() == '\\' || val.back() == '"')) {
00067             val.remove_suffix(1);
00068         }
00069
00070         if (!val.empty()) {
00071             sensor_ids.emplace_back(val);
00072         }
00073     }
00074
00075     return sensor_ids;
00076 }
00077
00078 } // namespace astralog
00079
00080 #endif // ASTRALOG_YAML_PARSER_HPP

```


Index

- ~MappedFile
 - astralog::MappedFile, 46
- add
 - astralog::RuleViolation, 61
- add_record
 - astralog::BatchAccumulator, 38
- alarm_lines
 - astralog::ProcessingResult, 54
- arr_val
 - astralog::detail::MiniJsonParser::JsonValue, 45
- ARRAY
 - astralog::detail::MiniJsonParser::JsonValue, 44
- as_int
 - astralog::detail::MiniJsonParser::JsonValue, 44
- as_number
 - astralog::detail::MiniJsonParser::JsonValue, 44
- as_string
 - astralog::detail::MiniJsonParser::JsonValue, 44
- assign_sensor_tokens
 - astralog, 15
- astralog, 11
 - assign_sensor_tokens, 15
 - BatchStrategy, 14
 - build_inverse_token_map, 15
 - build_line_offsets, 15
 - build_per_sensor_sequences, 16
 - build_timestamp_groups, 16
 - build_token_view_map, 17
 - CompOp, 14
 - CORRELATION, 15
 - COUNT, 14
 - EQ, 14
 - evaluate_comparison, 17
 - evaluate_correlations, 18
 - evaluate_rules_parallel, 19
 - evaluate_stateful, 20
 - evaluate_step_diff, 21
 - evaluate_threshold, 22
 - format_alarm_lines, 22
 - format_nominal_line, 23
 - format_value, 24
 - GE, 14
 - generate_output_parallel, 24
 - get_line, 26
 - GT, 14
 - HIGH, 14
 - LE, 14
 - load_rules, 26
 - load_sensors_from_yaml, 27
 - lookup_sensor_token, 28
 - LOW, 14
 - LT, 14
 - MEDIUM, 14
 - NE, 14
 - parse_csv_parallel, 28
 - parse_value, 30
 - prepare_rules, 30
 - Priority, 14
 - priority_to_string, 31
 - process_pipeline, 31
 - RuleType, 14
 - ruletype_to_string, 32
 - sort_by_priority, 32
 - sort_violations_for_output, 33
 - STATEFUL, 15
 - STEP_DIFF, 15
 - strategy_to_string, 33
 - string_to_compop, 33
 - string_to_priority, 34
 - string_to_ruletype, 34
 - string_to_strategy, 34
 - THRESHOLD, 14
 - TIME, 14
 - tokenize_csv_line, 35
 - TokenViewMap, 13
 - validate_schema, 35
- astralog::BatchAccumulator, 37
 - add_record, 38
 - batch_count, 39
 - BatchAccumulator, 38
 - buffered_count, 39
 - config, 39
 - flush, 39
 - flush_remaining, 40
 - should_flush, 40
- astralog::BatchConfig, 41
 - batch_output_dir, 42
 - BatchConfig, 42
 - count_threshold, 42
 - strategy, 42
 - time_threshold_ms, 42
 - write_audit_files, 43
- astralog::detail, 36
 - read_file_to_string, 36
- astralog::detail::MiniJsonParser, 47
 - parse, 48
- astralog::detail::MiniJsonParser::JsonValue, 43
 - arr_val, 45

- ARRAY, 44
- as_int, 44
- as_number, 44
- as_string, 44
- find, 44
- NONE, 44
- num_val, 45
- NUMBER, 44
- obj_val, 45
- OBJECT, 44
- str_val, 45
- STRING, 44
- Type, 44
- type, 45
- astralog::MappedFile, 45
 - ~MappedFile, 46
 - data, 47
 - fd, 47
 - open, 46
 - size, 47
- astralog::ParsedRecord, 48
 - ParsedRecord, 49
 - sensor_token, 49
 - timestamp, 49
 - value, 49
- astralog::PipelineState, 50
 - initialize, 50
 - initialized, 51
 - sensor_states, 51
- astralog::PreparedRules, 51
 - correlation_rules, 52
 - sensor_rules, 52
 - stateful_count, 52
 - step_diff_count, 52
 - threshold_count, 52
- astralog::ProcessingResult, 53
 - alarm_lines, 54
 - eval_ms, 54
 - num_anomalies, 54
 - num_nominal, 54
 - num_timestamps, 54
 - total_ms, 54
 - valid_lines, 54
- astralog::RuleDefinition, 56
 - consecutive_measurements, 57
 - logic, 57
 - op, 57
 - priority, 57
 - rule_id, 57
 - RuleDefinition, 57
 - sensor_id, 57
 - sensor_token_idx, 58
 - sub_rules, 58
 - threshold, 58
 - type, 58
- astralog::RuleViolation, 59
 - add, 61
 - count, 61
 - empty, 61
 - INLINE_CAPACITY, 61
 - rule, 62
 - RuleViolation, 60
 - sensor_token_at, 61
 - sensor_tokens_extra, 62
 - sensor_tokens_inline, 62
 - size, 61
 - value_at, 61
 - values_extra, 62
 - values_inline, 62
- astralog::SensorRuleSet, 62
 - stateful, 63
 - step_diff, 63
 - threshold, 63
- astralog::SensorRuleSet::StatefulEntry, 65
 - rule, 65
 - slot, 65
- astralog::SensorState, 63
 - consecutive_violations, 64
 - has_previous, 64
 - previous_value, 64
 - SensorState, 64
- astralog::TimestampGroup, 66
 - readings, 67
 - timestamp, 67
- astralog::TimestampGroup::Reading, 55
 - sensor_token, 55
 - value, 55
- astralog::TokenViewEntry, 67
 - sensor_id, 67
 - token, 67
- batch_count
 - astralog::BatchAccumulator, 39
- batch_output_dir
 - astralog::BatchConfig, 42
- BatchAccumulator
 - astralog::BatchAccumulator, 38
- BatchConfig
 - astralog::BatchConfig, 42
- BatchStrategy
 - astralog, 14
- buffered_count
 - astralog::BatchAccumulator, 39
- build_inverse_token_map
 - astralog, 15
- build_line_offsets
 - astralog, 15
- build_per_sensor_sequences
 - astralog, 16
- build_timestamp_groups
 - astralog, 16
- build_token_view_map
 - astralog, 17
- CompOp
 - astralog, 14
- config

- astralog::BatchAccumulator, 39
- consecutive_measurements
 - astralog::RuleDefinition, 57
- consecutive_violations
 - astralog::SensorState, 64
- CORRELATION
 - astralog, 15
- correlation_rules
 - astralog::PreparedRules, 52
- COUNT
 - astralog, 14
- count
 - astralog::RuleViolation, 61
- count_threshold
 - astralog::BatchConfig, 42
- csv_parser.hpp
 - MAP_POPULATE, 75
- data
 - astralog::MappedFile, 47
- empty
 - astralog::RuleViolation, 61
- EQ
 - astralog, 14
- eval_ms
 - astralog::ProcessingResult, 54
- evaluate_comparison
 - astralog, 17
- evaluate_correlations
 - astralog, 18
- evaluate_rules_parallel
 - astralog, 19
- evaluate_stateful
 - astralog, 20
- evaluate_step_diff
 - astralog, 21
- evaluate_threshold
 - astralog, 22
- fd
 - astralog::MappedFile, 47
- find
 - astralog::detail::MiniJsonParser::JsonValue, 44
- flush
 - astralog::BatchAccumulator, 39
- flush_remaining
 - astralog::BatchAccumulator, 40
- format_alarm_lines
 - astralog, 22
- format_nominal_line
 - astralog, 23
- format_value
 - astralog, 24
- GE
 - astralog, 14
- generate_output_parallel
 - astralog, 24
- get_line
 - astralog, 26
- GT
 - astralog, 14
- has_previous
 - astralog::SensorState, 64
- HIGH
 - astralog, 14
- initialize
 - astralog::PipelineState, 50
- initialized
 - astralog::PipelineState, 51
- INLINE_CAPACITY
 - astralog::RuleViolation, 61
- LE
 - astralog, 14
- load_rules
 - astralog, 26
- load_sensors_from_yaml
 - astralog, 27
- logic
 - astralog::RuleDefinition, 57
- lookup_sensor_token
 - astralog, 28
- LOW
 - astralog, 14
- LT
 - astralog, 14
- main
 - main.cpp, 80
- main.cpp
 - main, 80
- MAP_POPULATE
 - csv_parser.hpp, 75
- MEDIUM
 - astralog, 14
- NE
 - astralog, 14
- NONE
 - astralog::detail::MiniJsonParser::JsonValue, 44
- num_anomalies
 - astralog::ProcessingResult, 54
- num_nominal
 - astralog::ProcessingResult, 54
- num_timestamps
 - astralog::ProcessingResult, 54
- num_val
 - astralog::detail::MiniJsonParser::JsonValue, 45
- NUMBER
 - astralog::detail::MiniJsonParser::JsonValue, 44
- obj_val
 - astralog::detail::MiniJsonParser::JsonValue, 45
- OBJECT
 - astralog::detail::MiniJsonParser::JsonValue, 44

- op
 - astralog::RuleDefinition, 57
- open
 - astralog::MappedFile, 46
- parse
 - astralog::detail::MiniJsonParser, 48
- parse_csv_parallel
 - astralog, 28
- parse_value
 - astralog, 30
- ParsedRecord
 - astralog::ParsedRecord, 49
- prepare_rules
 - astralog, 30
- previous_value
 - astralog::SensorState, 64
- Priority
 - astralog, 14
- priority
 - astralog::RuleDefinition, 57
- priority_to_string
 - astralog, 31
- process_pipeline
 - astralog, 31
- read_file_to_string
 - astralog::detail, 36
- readings
 - astralog::TimestampGroup, 67
- rule
 - astralog::RuleViolation, 62
 - astralog::SensorRuleSet::StatefulEntry, 65
- rule_id
 - astralog::RuleDefinition, 57
- RuleDefinition
 - astralog::RuleDefinition, 57
- RuleType
 - astralog, 14
- ruletype_to_string
 - astralog, 32
- RuleViolation
 - astralog::RuleViolation, 60
- sensor_id
 - astralog::RuleDefinition, 57
 - astralog::TokenViewEntry, 67
- sensor_rules
 - astralog::PreparedRules, 52
- sensor_states
 - astralog::PipelineState, 51
- sensor_token
 - astralog::ParsedRecord, 49
 - astralog::TimestampGroup::Reading, 55
- sensor_token_at
 - astralog::RuleViolation, 61
- sensor_token_idx
 - astralog::RuleDefinition, 58
- sensor_tokens_extra
 - astralog::RuleViolation, 62
- sensor_tokens_inline
 - astralog::RuleViolation, 62
- SensorState
 - astralog::SensorState, 64
- should_flush
 - astralog::BatchAccumulator, 40
- size
 - astralog::MappedFile, 47
 - astralog::RuleViolation, 61
- slot
 - astralog::SensorRuleSet::StatefulEntry, 65
- sort_by_priority
 - astralog, 32
- sort_violations_for_output
 - astralog, 33
- src Directory Reference, 9
- src/batch_accumulator.hpp, 69, 71
- src/csv_parser.hpp, 73, 76
- src/main.cpp, 79
- src/output_formatter.hpp, 81, 83
- src/rules_engine.hpp, 84, 86
- src/timestamp_processor.hpp, 90, 93
- src/types.hpp, 99, 101
- src/yaml_parser.hpp, 104, 105
- STATEFUL
 - astralog, 15
- stateful
 - astralog::SensorRuleSet, 63
- stateful_count
 - astralog::PreparedRules, 52
- STEP_DIFF
 - astralog, 15
- step_diff
 - astralog::SensorRuleSet, 63
- step_diff_count
 - astralog::PreparedRules, 52
- str_val
 - astralog::detail::MiniJsonParser::JsonValue, 45
- strategy
 - astralog::BatchConfig, 42
- strategy_to_string
 - astralog, 33
- STRING
 - astralog::detail::MiniJsonParser::JsonValue, 44
- string_to_compop
 - astralog, 33
- string_to_priority
 - astralog, 34
- string_to_ruletype
 - astralog, 34
- string_to_strategy
 - astralog, 34
- sub_rules
 - astralog::RuleDefinition, 58
- THRESHOLD
 - astralog, 14
- threshold

- astralog::RuleDefinition, [58](#)
 - astralog::SensorRuleSet, [63](#)
- threshold_count
 - astralog::PreparedRules, [52](#)
- TIME
 - astralog, [14](#)
- time_threshold_ms
 - astralog::BatchConfig, [42](#)
- timestamp
 - astralog::ParsedRecord, [49](#)
 - astralog::TimestampGroup, [67](#)
- token
 - astralog::TokenViewEntry, [67](#)
- tokenize_csv_line
 - astralog, [35](#)
- TokenViewMap
 - astralog, [13](#)
- total_ms
 - astralog::ProcessingResult, [54](#)
- Type
 - astralog::detail::MiniJsonParser::JsonValue, [44](#)
- type
 - astralog::detail::MiniJsonParser::JsonValue, [45](#)
 - astralog::RuleDefinition, [58](#)
- valid_lines
 - astralog::ProcessingResult, [54](#)
- validate_schema
 - astralog, [35](#)
- value
 - astralog::ParsedRecord, [49](#)
 - astralog::TimestampGroup::Reading, [55](#)
- value_at
 - astralog::RuleViolation, [61](#)
- values_extra
 - astralog::RuleViolation, [62](#)
- values_inline
 - astralog::RuleViolation, [62](#)
- write_audit_files
 - astralog::BatchConfig, [43](#)